SECJ2203: Software Engineering

System Documentation (SD)

Software Requirements Specification (SRS)

Hast Travel Car Rental

Version 1.0

Date: 17/11/2025

Faculty of Computing

Prepared by: Hephaestus

Revision Page

a. Overview

The current 1.0 version of the System Documentation (SD) includes Chapter 1 Introduction and Chapter 2 Specific Requirements of the Hasta Car Booking System. This SD describes the Software Requirements Specification (SRS) phase of the project. The Introduction chapter contains purpose, scope, definition, acronyms, and abbreviation, references and overview of the system. At the same time, the Specific Requirements chapter covers the detailed personas, system features, launch phase, user stories involving system models such as use case diagram, domain model and state machine diagram. The chapter also focuses on performance and other requirements as well as design constraints of the proposed system.

b. Target Audience

This SD is intended for individuals and teams involved in the design, development, and evaluation of the Hasta Car Booking System. The main audience includes the project stakeholders, academic supervisor and system development team members. This documentation serves as a reference for the project team members to ensure consistent understanding of the system requirements and design specifications throughout the development process.

c. Project Team Members

The project team divided the SRS project documentation parts for each member to handle. Each member is assigned to specific tasks to ensure the documentation process is undergone efficiently and effectively. The table below shows the overview of team members, roles, task handles and the status of the tasks.

Member Name	Role	Task	Status

Muhammad Hafiz Bin Reepei	Group Leader	• Purpose
		• 1.2 Scope
		• 2.2 System Feature
		  - Use Case Diagram
		• 2.4 User Story Detail
		  - 2.4.1 User story Detail 1
		  - 2.4.2 User story Detail 2
		  - 2.4.3 User story Detail 3
		  - 2.4.4 User story Detail 4
		  - 2.4.20 User story Detail 20
		• 2.5 Performance and Other Requirements
		  - Performance
			Completed

Ahmad Irfan Bin Azahan	Group Member	• 2.1 Persona - 2.1.1 Persona 1: Customer - 2.1.2 Persona 2: Staff - 2.1.3 Persona 3: Administrator • 2.2 System Feature - State Machine Diagram • 2.4 User Story Detail - 2.4.5 User story Detail 5 - 2.4.6 User story Detail 6 - 2.4.7 User story Detail 7 - 2.4.8 User story Detail 8 - 2.4.9 User story Detail 9 - 2.4.10 User story Detail 10 • 2.5 Performance and Other Requirements - Other Requirement	Completed
Cheong Yi Shien	Group Member	• Revision Page - Overview - Target Audience - Project Team Members - Version Control History • 2.3 Launch phase • 2.4 User Story Detail - 2.4.11 User story Detail 11 - 2.4.12 User story Detail 12 - 2.4.13 User story Detail 13 - 2.4.14 User story Detail 14 - 2.4.15 User story Detail 15	Completed

Ling Yu Qian	Group Member	• 1.3 Definitions, Acronyms and Abbreviations
		• 1.4 Reference
		• 1.5 Overview
		• 2.2 System Feature
		  - Class Diagram
		• 2.4 User Story Detail
		  - 2.4.16 User story Detail 16
		  - 2.4.17 User story Detail 17
		  - 2.4.18 User story Detail 18
		  - 2.4.19 User story Detail 19
		• 2.6 Design constraint
			Completed

d. Version Control History

The Version Control History records all modifications made to this SRS document, including updates to requirements, corrections, and additions. Each entry notes the version number, date, author, and a brief description of the changes. This helps maintain clarity and traceability of revisions over time. The table below summarizes the version history of the document.

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Team leader 1 (Muhammad Hafiz Bin Reepei)	Completed Version 1.0 including Chapter 1 until Chapter 2, Section 2.6	29/11/2025

Table of Contents

1. Introduction

1.1 Purpose

This SD provides all the details related to the requirements, design, and documentation for developing the Hasta Travel Car Rental system. The purpose of this System Documentation (SD) is to put into place a clear and unified reference that shall guide the analysis, design, implementation, and evaluation of the software during its development life cycle. It includes the functional and non-functional requirements of the system, user characteristics, models of the system, constraints, and general expectations from the operation of the system for uniformity among the various contributors. The SD is targeted at project stakeholders such as the academic supervisor, the developers, designers, and testers of the system, who depend on this document for an understanding of the system objectives to realize the intended goals in the final output of their work. It also serves to communicate among decision-makers and evaluators about the completeness, quality, and appropriateness regarding specified requirements.

1.2 Scope

The software product to be developed is the Hasta Car Booking System, a customized web-based car rental administration platform designed specifically for Hasta Travel to replace its current semi-manual and error-prone workflow. The system is built using Laravel as the primary backend framework and MySQL as the database management system, supported by HTML, CSS, and JavaScript for the front-end interface. It is designed to centralize and automate key operations such as online booking, document submission, payment proof upload, real-time vehicle availability tracking, booking approval, and administrative reporting. The system will not integrate external payment gateways in this phase; instead, it will rely on a manual proof-of-payment upload mechanism to control development costs and scope. The application will benefit the organization by reducing redundancy, eliminating miscommunication, improving staff productivity, and enhancing customer satisfaction through faster processing, clearer workflows, and greater transparency. Additionally, the system provides supervisors with consolidated data and analytics to support data-driven decision-making. This scope is consistent with the NABC proposal, where Laravel and MySQL were identified as cost-effective, open-source technologies suitable for delivering an efficient, scalable, and reliable digital solution tailored to Hasta Travel's operational needs.

1.3 Definitions, Acronyms and Abbreviation

Term	Definition
Constraint	Limitation or restriction on system behavior and resources.
CRUD	Create, Read, Update, and Delete, the basic operation for managing data in the system.
SD	System Documentation
SRS	Software Requirement Specification
Invoice/Receipt	Digital document generated by system as a proof of payment for a booking
Sprint	A fixed length of period in Agile development to complete the specific work within period and ready to review the completed work.
Dashboard	A centralized user interface that provides an overview of relevant information.

1.4 References

Nuñez, R. (2014, October 13). *UML - and the Unified Process*. Academia.edu. https://www.academia.edu/8751505/UML_and_the_Unified_Process

Software Requirements | Software Engineering Tutorial | Minigranth. (2022). Minigranth.in. https://minigranth.in/software-engineering-tutorial/software-requirements-sdlc

Sommerville, I. (2016). *Software Engineering Tenth Edition*. https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf

UML 2 Sequence Diagrams: An Agile Introduction – *The Agile Modeling (AM) Method*. (n.d.). https://agilemodeling.com/artifacts/sequencediagram.htm

1.5 **Overview**

This system documentation is organized into two chapters. The first chapter is an introduction that contains purpose, scope, and definitions for the projects. The second chapter includes persona, system features, user stories with diagrams, performance, requirements, and constraints of the system.

2. Specific Requirements

This section defines all the software requirements of the Hasta Travel Car Rental system in detail to allow system designers to design the system and testers to verify that it meets the specified requirements. These requirements include all actions that the system must perform in response to user inputs or requests from other systems, as well as the outputs the system must produce. By clearly defining what the system should do in observable ways, this section ensures that the system's functionality can be understood, implemented, and tested effectively, providing confidence that the final product meets user needs and expectations.

2.1 Persona

This section describes the intended users of the Hasta Travel Car Rental system and their key characteristics. Understanding these users helps the development team design and develop a system that meets their needs, is easy to use, and is accessible to the users.

2.1.1 Persona 1: Customer

User Need

Name: Umar bin Abu

Age: 20

Knowledge: Tech-savvy and comfortable with online booking services. He is frustrated by the current semi-manual process of renting cars, which requires him to use online forms that redirect to WhatsApp for confirmation. He finds this fragmented communication inefficient and slow.

User Stories

1) As a customer, I want to register an account so that I can access the booking system.
2) As a customer, I want to log in to my account securely so that I can manage my bookings.
3) As a customer, I want to view my booking history so that I can review my past and upcoming rentals.
4) As a customer, I want to browse all available cars, so that I can choose a suitable vehicle for my booking.
5) As a customer, I want to check real time car's availability for a selected date and time so that I can plan my booking.
6) As a customer, I want to book a selected car with my preferred date and duration so that I ca plan my travel efficiently.
7) As a customer, I want to cancel my booking if my plans change so that I do not occupy a vehicle I would not use.

8) As a customer, I want to make payments for my booking through the system so that my reservation is confirmed.
9) As a customer, I want to view or download my receipt or invoice so that I have proof of payment.

2.1.2 Persona 2: Staff

Name: Fatin binti Adam

Age: 30

Knowledge: He is an administrative staff member who currently manages bookings using the semi-manual process. He is frustrated by the constant switching between the web form, WhatsApp, and bank statements to confirm a single booking.

User Need

An admin staff member needs a centralized dashboard to efficiently manage all booking requests, track vehicle availability in real-time, and automate the payment verification and invoicing process. This is to reduce administrative errors, speed up customer confirmations, and maintain an accurate, consolidated record of all transactions.

User Stories

1) As a staff member, I want to log into the system so that I can perform booking-related functions.
2) As a staff member, I want to check car availability so that I can ensure the on-time availability is always match to the booking lists.
3) As a staff member, I want to verify incoming payments so that bookings are confirmed only when payment is valid
4) As a staff member, I want to review and approve or reject cancellation requests submitted by customers, so that the booking record, refund amount, and car availability can be updated correctly.
5) As a staff member, I want the system to generate invoices and receipts after payment verification so that customers receive official records and the accounting is accurate.
6) As a staff member, I want to add, delete, read, and update vehicle records new vehicle records so that the car owner can register for their car for rent.

2.1.3 Persona 3: Administrator

Name: Puan Haslinda binti Othman

Age: 48

Knowledge: As the manager of Hasta Travel, she is focused on strategic growth and profitability. She lacks access to consolidated data, which prevents her from making data-driven decisions to improve revenue. She needs to monitor overall performance, not just individual bookings.

User Need

A administrator needs a high-level, real-time view of the business's performance through a centralized system. This is to analyze sales trends, monitor staff efficiency, and access customer feedback to make strategic decisions that improve sales and customer satisfaction.

User Stories

1) As an administrator, I want to log into the system so that I can access system-wide management functions.
2) As an administrator, I want to add, update, and delete vehicle records, so that I can maintain an accurate list of cars available for booking.
3) As an administrator, I want to generate rental and booking reports, so that I can review system performance, rental trends, and peak session.
4) As an administrator, I want to view reports in charts and summaries, so that I can make data-driven decisions efficiently.

2.2 System Features

The Hasta System Travel Car Rental is a software application designed to operate on both web and mobile platforms. The system provides a convenient way for users to book, manage, and cancel car reservations, while also enabling administrators to monitor bookings and maintain the system efficiently. The system is integrated with a secure, cloud-based database to ensure real-time updates for bookings, cancellations, and user account management. Payment processing is also handled through a reliable external gateway for secure transactions.

The system is designed to serve three main types of users: regular users and system administrators. Figure 1.1 shows the use case diagram for the Hasta Travel Car Rental System which shows the interaction between the user and the system. The detailed use case descriptions in each module are explained in Table 1.0. The domain model diagram in Figure 1.2 is the class diagram showing the classes in the system and associations between the classes while Figure 1.3 shows the state diagram of Hasta Travel car Rental System which shows the different states of the system and how it transitions between the states based on conditions.

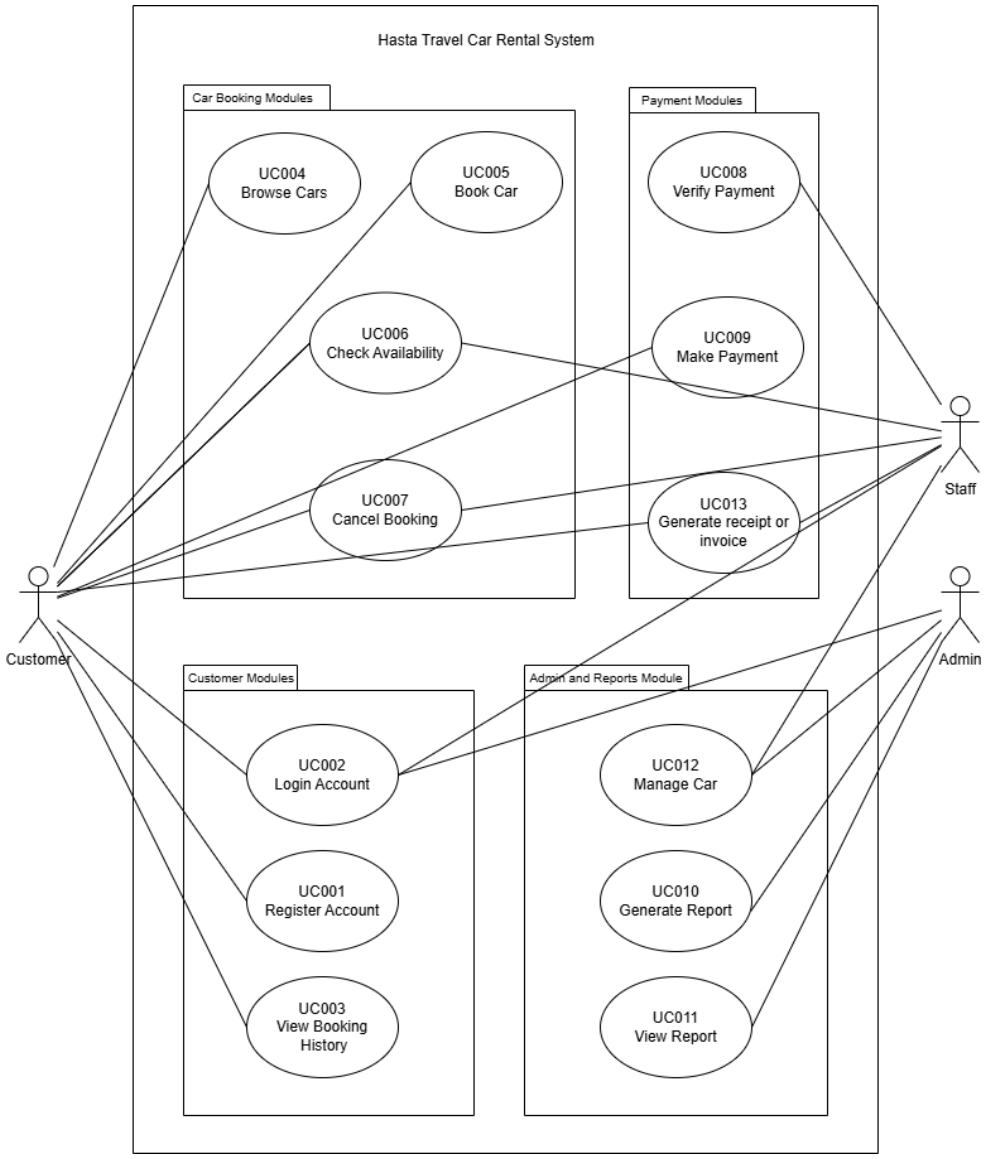

Figure 1.1: Use Case Diagram for <Hasta Travel Car Rental>

Table 1.0: Description of Module and Functions for <Hasta Travel Car Rental>

Use case	Function	Description
UC001	Register Account	Allows user to create new account in system.
UC002	Login Account	Allows users to book the car in the system
UC003	View Booking History	To see a list of all past and current car rental bookings.
UC004	Browse Car	View list of available cars, filtered and sorted as needed
UC005	Book Car	To select available car, fill details and do payment to book.
UC006	Check Availability	The system displays all available car based on selected dates and times
UC007	Cancel Booking	To cancel the existing car rental booking and triggers applicable refunds.
UC008	Verify Payment	To confirm payment transaction was successful legitimate.
UC009	Make Payment	To pay and confirm car rental booking.
UC010	Generate Report	To create report based on specific criteria.
UC011	View Report	To display previously generated reports.
UC012	Manage Car	To perform CRUD on the car inventory of the system.
UC013	Generate receipt/invoice	To generate a digital invoice/receipt once the customer's payment is successfully verified.

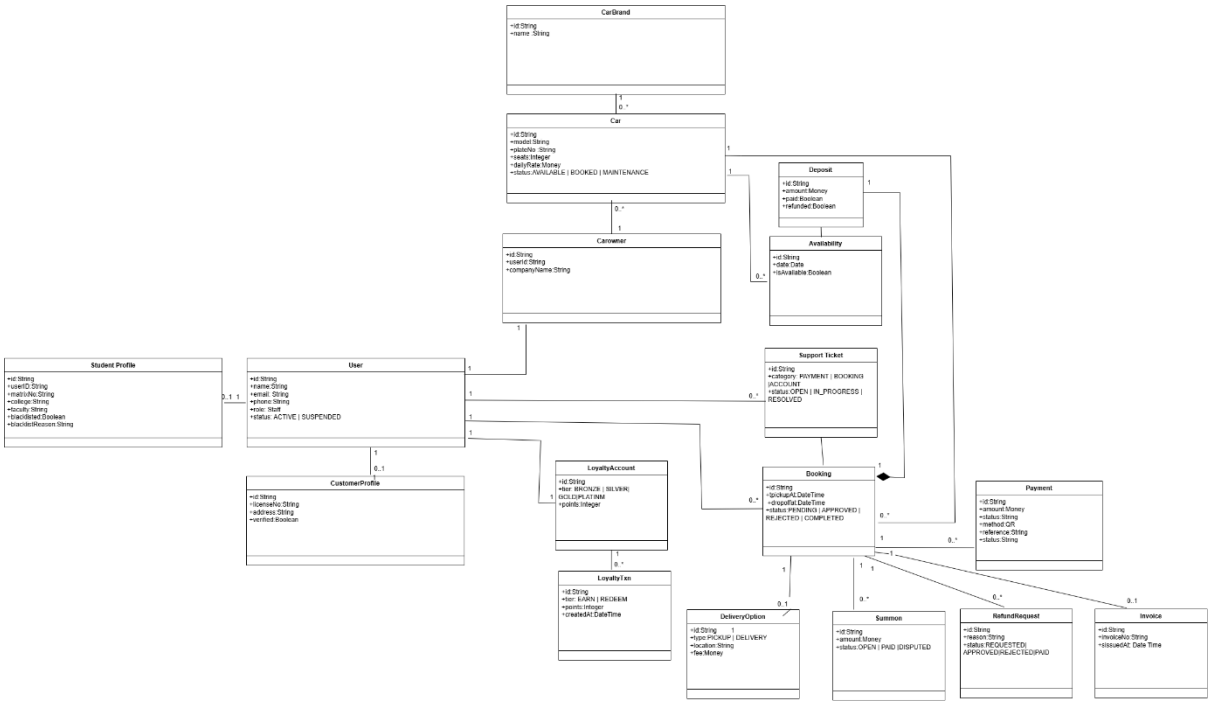

Figure 1.2: Domain Model for <Hasta Travel Car Rental>

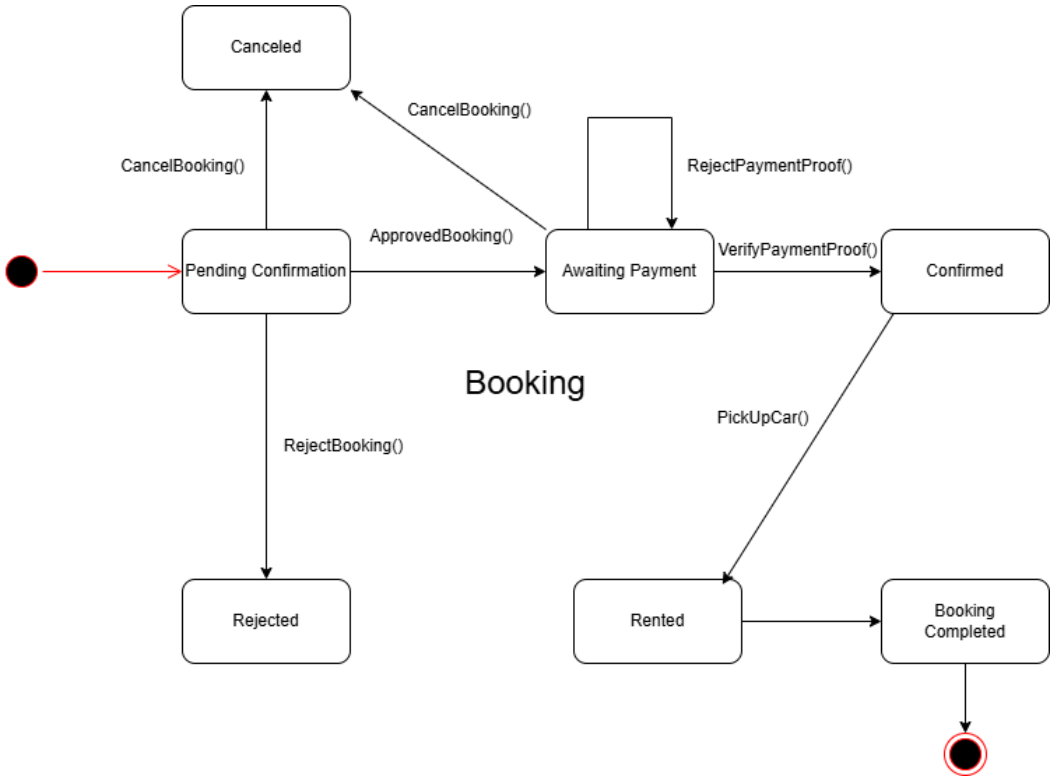

Figure 1.3: State Machine Diagram for Hasta Travel Car Rental

2.3 Launch Phase

The launch phase highlights the Agile development plan for Hasta Travel Car Rental system based on sprints involved during system development. This part includes the product backlogs, sprint planning and team members in charge to ensure development works are organized and completed iteratively to deliver functional increments of the system.

The product backlog lists all features, enhancements and system requirements needed for Hasta Car Booking System. Items in the backlog are derived from personas, user stories, and system features identified in earlier sections. Table below shows the sprint plan with user stories and team members assigned.

Sprint	**Team members assigned**
Sprint #1: Vehicle Management Module UC012: Manage car	Muhammad Hafiz Bin Reepei
Sprint #2: Car Booking Module UC004: Browse Car UC005: Book Car UC006: Check Availability	Muhammad Hafiz Bin Reepei
Sprint #3: Customer Module UC001: Register Account UC002: Login Account UC003: View Booking History	Ahmad Irfan Bin Azahan
Sprint #4: Booking Design Module UC005 Book Car UC007: Cancel Booking	Ahmad Irfan Bin Azahan
Sprint #5: Admin Module: UC012: Manage car	Cheong Yi Shien
Sprint #6: Reporting Module UC010: Generate Report UC011: View Report	Cheong Yi Shien
Sprint #7: Booking Cancellation Module UC007: Cancel Booking	Cheong Yi Shien

Sprint #8: Payment Module UC008: Verify Payment UC009: Make Payment	Ling Yu Qian
Sprint #9: Billing Module UC013: Generate Invoice/Receipt	Ling Yu Qian

2.4 **User Story Details**

This section provides details of user stories for the Hasta Travel Car Rental system. Each of the user stories include flow of event, alternative flow, acceptance criteria, exception flow, sequence diagram and activity diagram are documented to ensure clear understanding and accurate implementation.

2.4.1 US001: User Story < Staff > Sprint 1 – Vehicle Management Module

Table 2.1 depicts User Story Description for <Manage Car> use case. Figure 2.1.1 describes step-by-step interaction of staff with systems to manage cars based on Table 2.1. The staff are required to be logged into system first and can proceed to choose between adding, delete, read, or update the vehicle records to manage the car. Figure 2.1.2 illustrates the workflow and decisions for staff to interact accordingly to add, delete, read, or update vehicles records.

Table 2.1: User Story Description for <Manage Car>

User story: <Manage Car>
ID: US001
User Story Description **As a** staff member, **I want** to add, delete, read, and update vehicle records **So that** the car owner can register for their car for rent.
Flow of events: 1. Staff logs into the system 2. Staff navigates to Car Management section 3. Staff can select one of following options: • Add Car • Edit Car • Delete Car 4.If Add Car: • Staff enter car details • Staff clicks Save to store new car record. 5.If Edit Car: • Staff selects existing car. • Staff updates required fields. • Staff clicks Update. 6.If Delete Car: • Staff selects existing car. • Staff clicks Delete and confirms deletion. 7.Display updated car list
Alternative flow: Staff click Cancel, system will navigate to car list without changes.
Acceptance Criteria Post-condition: • Car list updated Pre-condition: • Staff is logged into the system.

Exception flow:

1. If the picture of the car is not loaded, the system will prompt “Please upload a PNG/JPEG file”.
2. If the staff cannot save the car details, the system prompts “Unable to save car details. Please try again.”

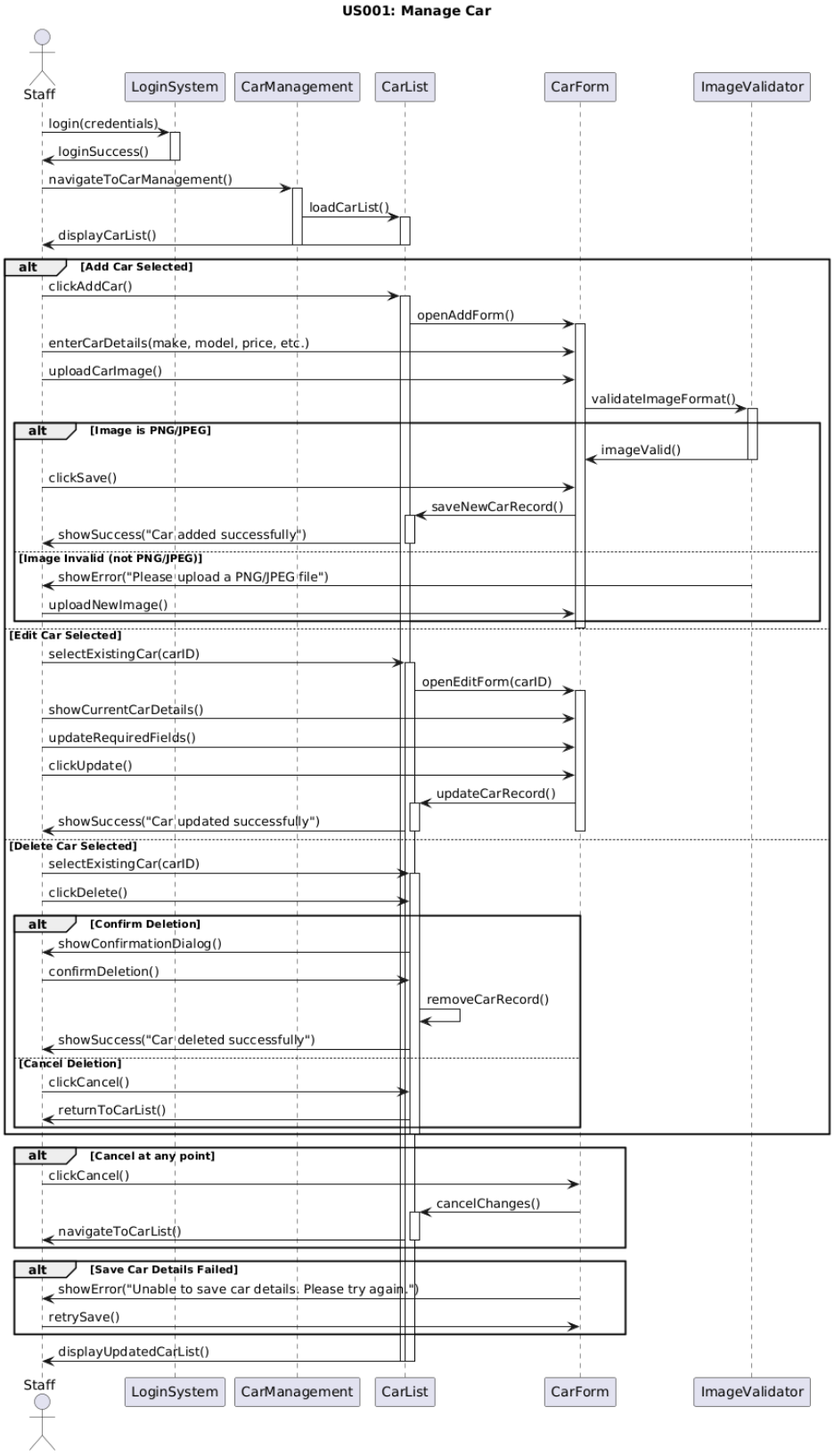

Figure 2.1.1: Sequence Diagram for <Manage Car>

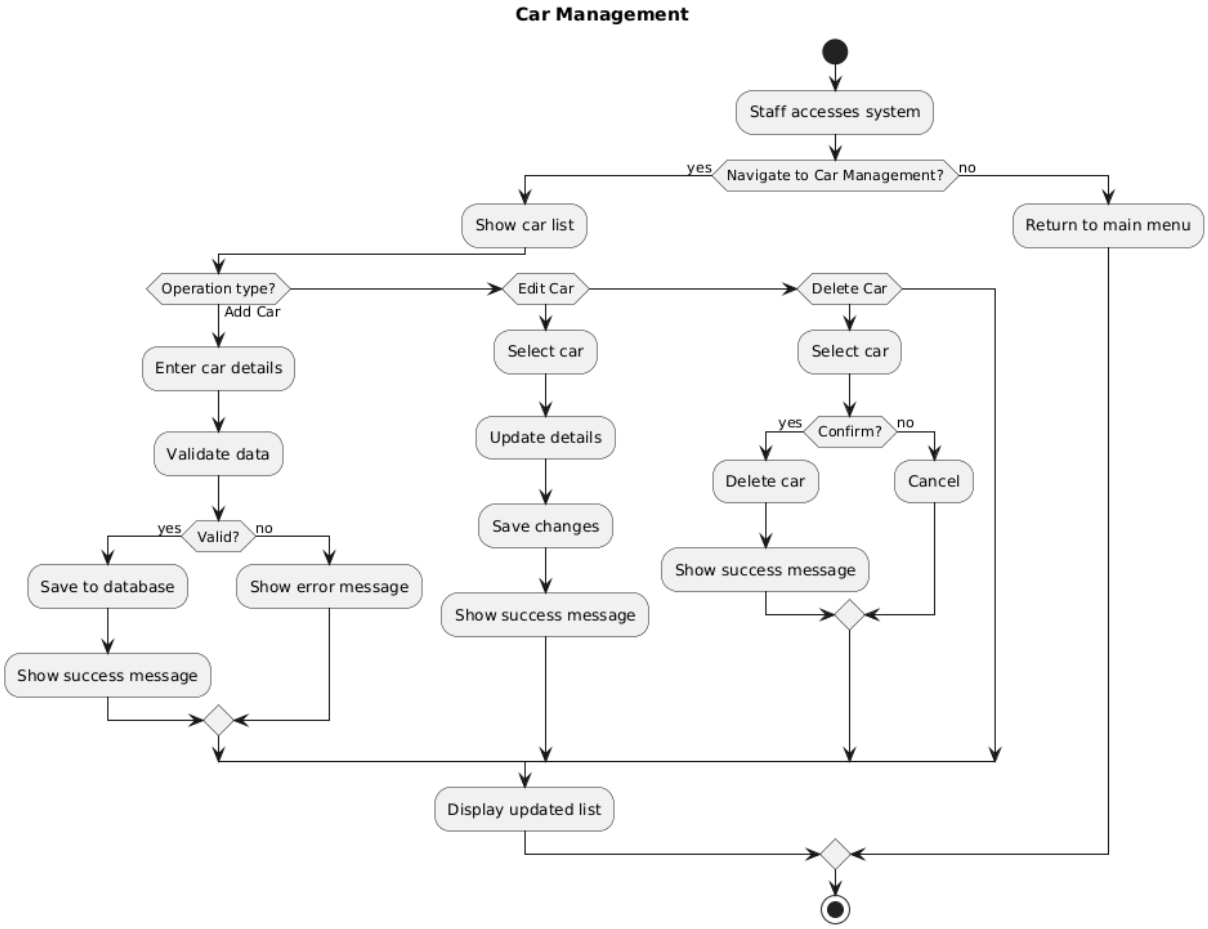

Figure 2.1.2: Activity Diagram for <Manage Car>

2.4.2 US002: User Story < Customer > Sprint 2 – Car Booking Module

Table 2.2 presents User Story Description for <Browse Car> use case. The table describes flows that customer can browse the car in the system. Figure 2.2.1 shows and Figure 2.2.2 illustrate the Sequence Diagram and Activity Diagram according to the Table 2.2 flows to browse car as a customer. Customer can browse the car by default at the home page of the system or can apply filter features for specific search.

Table 2.2: User Story Description for <Browse Car>

User story: <Browse Car>
ID: US002
User Story Description **As a** customer, **I want** to browse all available cars, **So that** I can choose a suitable vehicle for my booking.
Flow of events: 1. Customer opens website homepage. 2. Customer clicks Browse Car. 3. System display list of cars. 4. Customer can scroll through the list or view car details. 6. System refreshes the list after filters.
Alternative flow: 1. Customer can apply filters based on dates, prices, car type, and more to browse more specific. If no cars available, the system displays “No cars available at the moment”
Acceptance Criteria Post-condition: • Customers can view car details without logging in the system. Pre-condition: • Cars must exist in the system.
Exception flow: If the system fails to load car list, the system displays “Unable to load car list. Please try again later.”

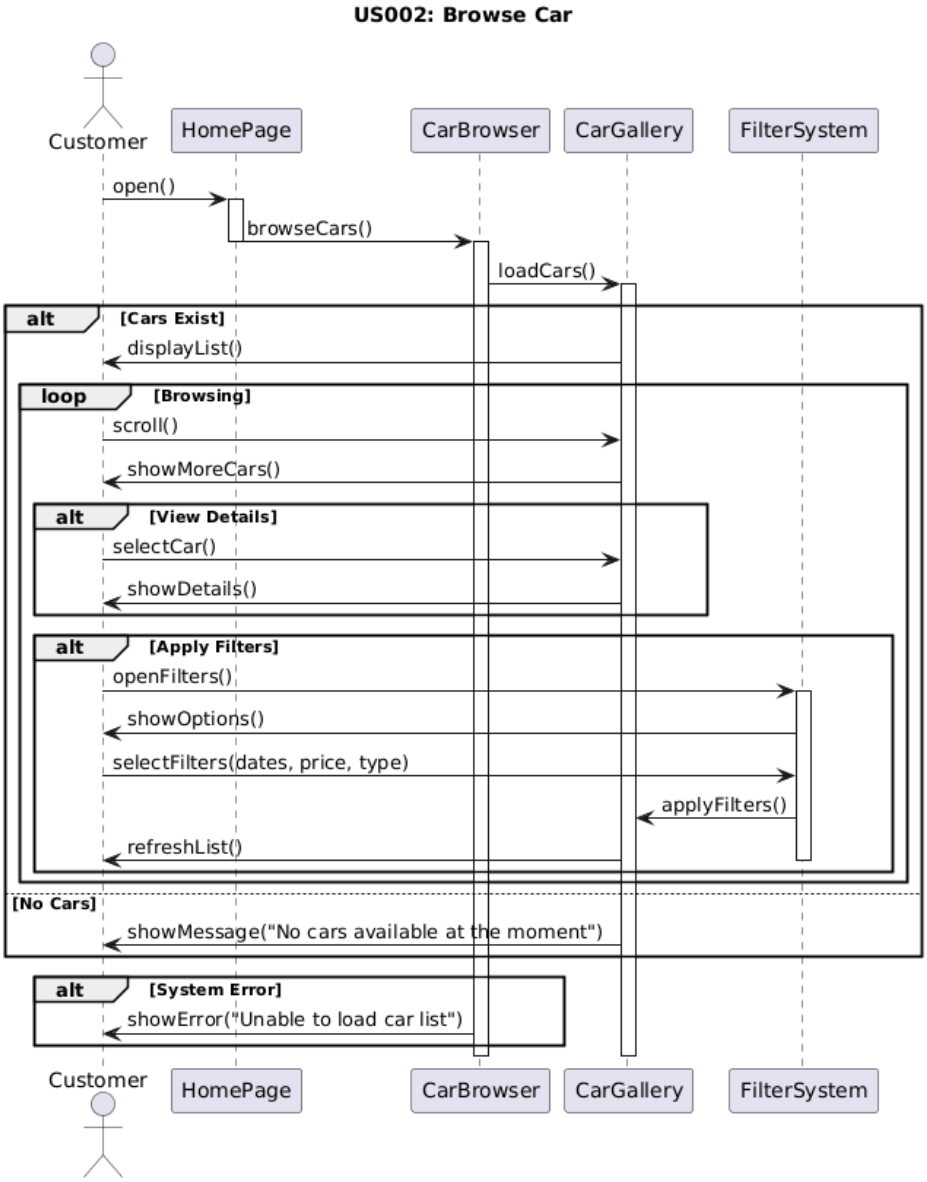

Figure 2.2.1: Sequence Diagram for <Browse Car>

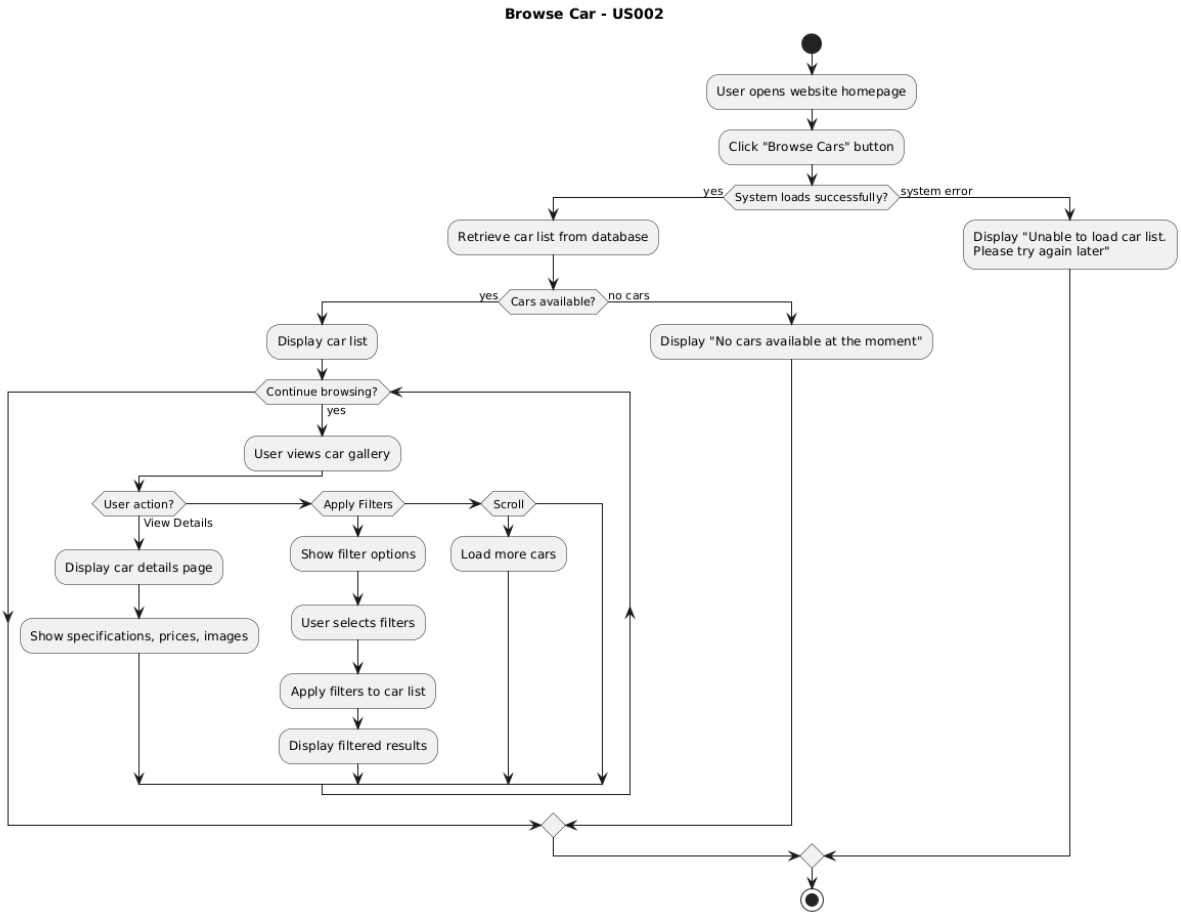

Figure 2.2.2: Activity Diagram for <Browse Car>

2.4.3 US003: User Story < Customer > Sprint 2 – Car Booking Module

Table 2.3 shows User Story Description for <Book Carr> use case of flow events for customers to interact in the system to book a car. Figure 2.3.1 and Figure 2.3.2 provides clarity process on how customers can interact with the system to book a car. The customer required to fill out the form first, before proceeding to payment. A message will display to prevent proceeding from payment if one of the required details is incomplete.

Table 2.3: User Story Description for <Book Car>

User story: Book Car
ID: US003
Customer Story Description **As a** customer, **I want** to book a selected car with my preferred date and duration **So that** I can plan my travel efficiently.
Flow of events: 1. Customer logs into the system. 2. Customer selects a car and views car details. 3. Customer selects preferred date and duration and clicks Book Now. 3. Customer fills out the booking form and clicks Submit Booking. 4. System displays selected car information, pickup date, rental duration, deposit, and rental price. 5. System displays instructions to proceed with payment.
Alternative flow: If the Customer changes their mind or picks the wrong date, they can click Cancel and the system will return to the car details page.
Acceptance Criteria Post-condition: • Booking records will be stored in the system; Customer will proceed to payment. Pre-condition: • The car must be available within the selected date range.
Exception flow: System prompts Customer to complete the details if the Customer missed fill the required fields.

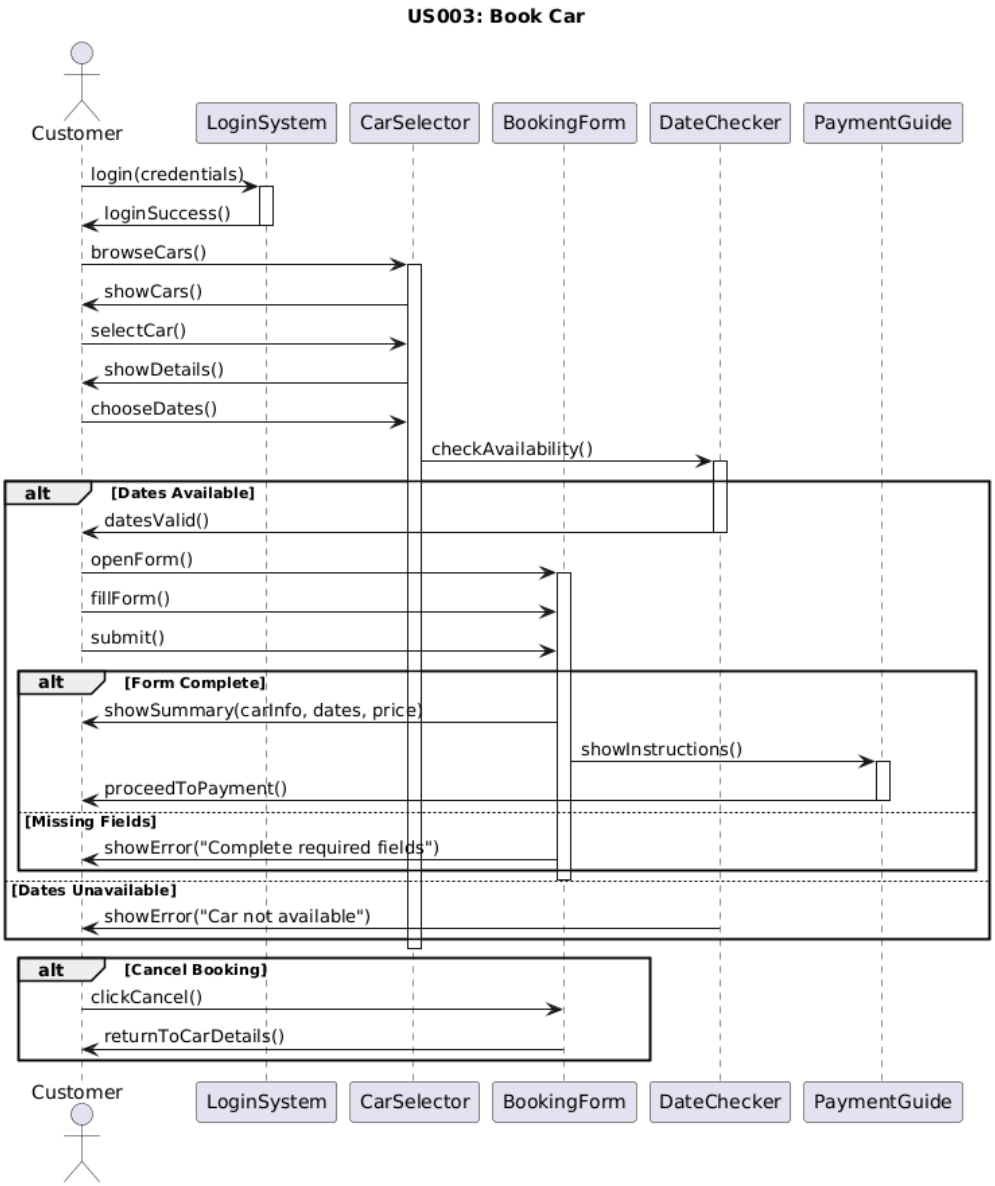

Figure 2.3.1: Sequence Diagram for <Book Car>

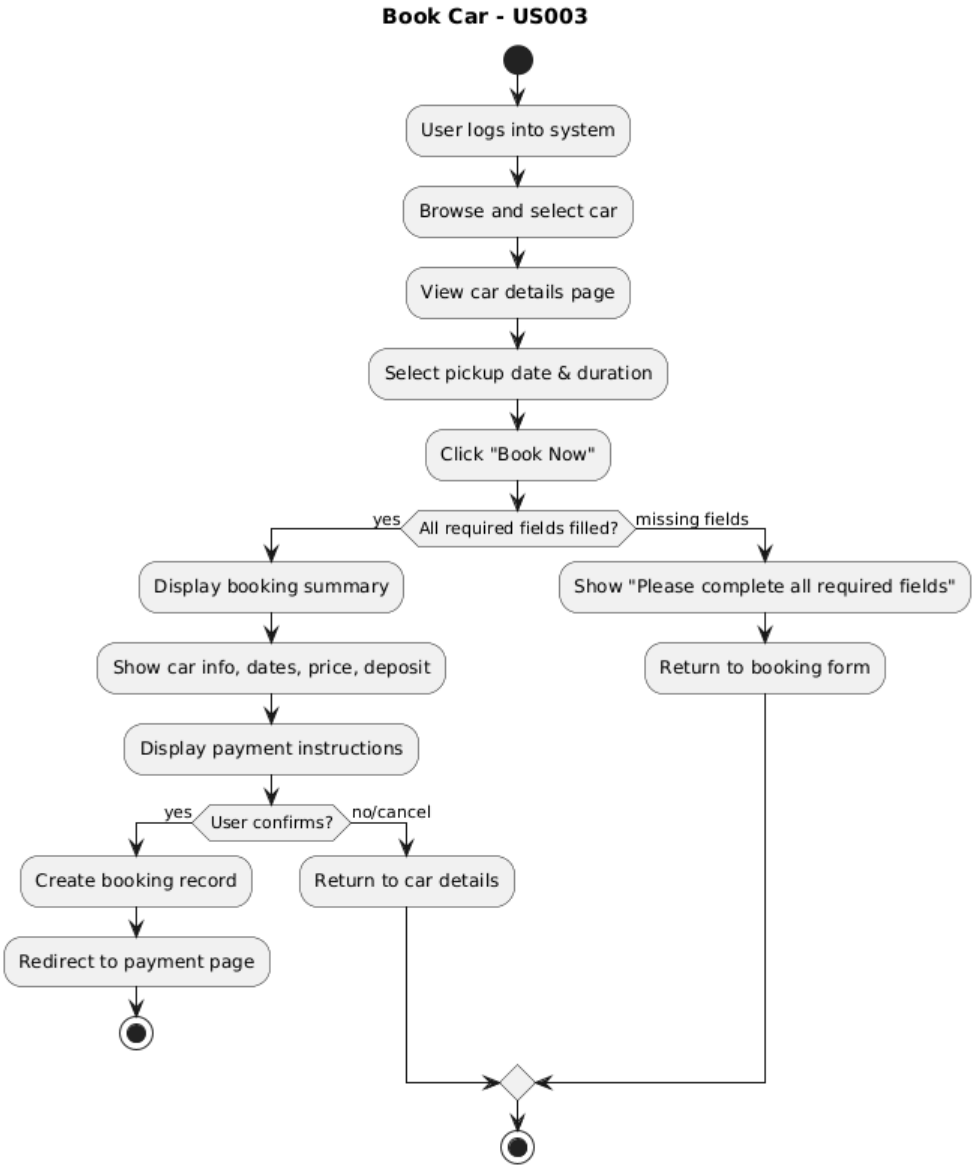

Figure 2.3.2: Activity Diagram for <Book Car>

2.4.4 US003: User Story < Customer > Sprint 2 – Car Booking Module

Table 2.4 provides User Story Description for <Check Availability> use case of flow events for customers to view availability of cars. Figure 2.4.1 shows that customers can check the availability of a car with or without logging in to the system. Figure 2.4.2 visualizes the decisions that can be made by customers according to Table 2.4 flows to check the availability of cars.

Table 2.4: User Story Description for Check Availability

User story: Check Availability
ID: US004
Customer Story Description **As a** customer, **I want** to check real-time car availability for a selected date and time **So that** I can plan my booking.
Flow of events: 1. Customer opens the web homepage. 2. Customer selects a car and clicks view details. 3. Customer selects the desired start date and end date. 4. The system disables (blurs, grey out) the dates that are already booked or unavailable, preventing Customers from selecting the date. 5. If the selected range date is overlapped or unavailable, then the system prompts the Customer to select different dates. 6. Customers select valid dates. 7. Customer proceeds to booking form
Alternative flow: 1. Customer opens the web homepage. 2. Customer uses a date filter to select the desired start date and end date. 3. System displays list of cars that only available within the range date
Acceptance Criteria Post-condition: • Customer receives availability status and proceeds to booking Pre-condition: none
Exception flow: If fate input is missing, the system prompts the Customer to select dates.

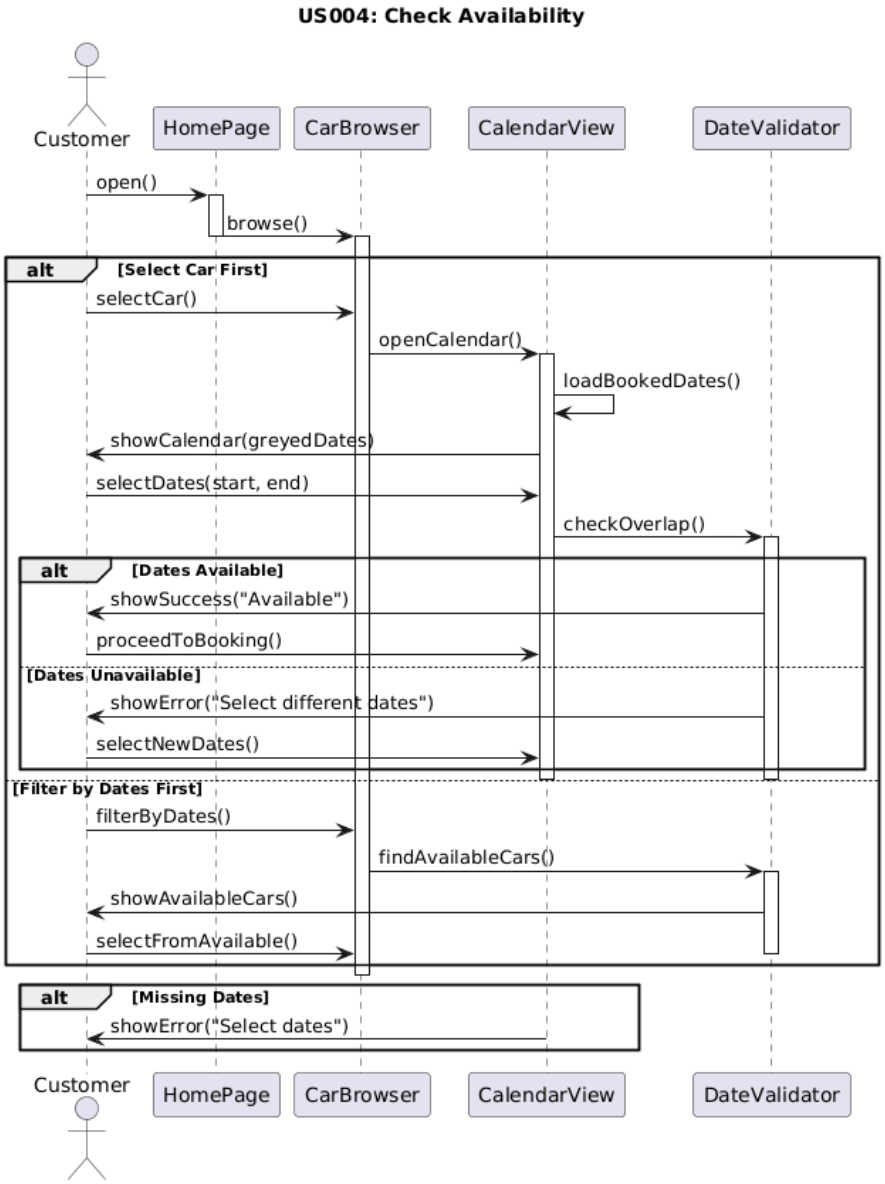

Figure 2.4.1: Sequence Diagram for <Check Availability>

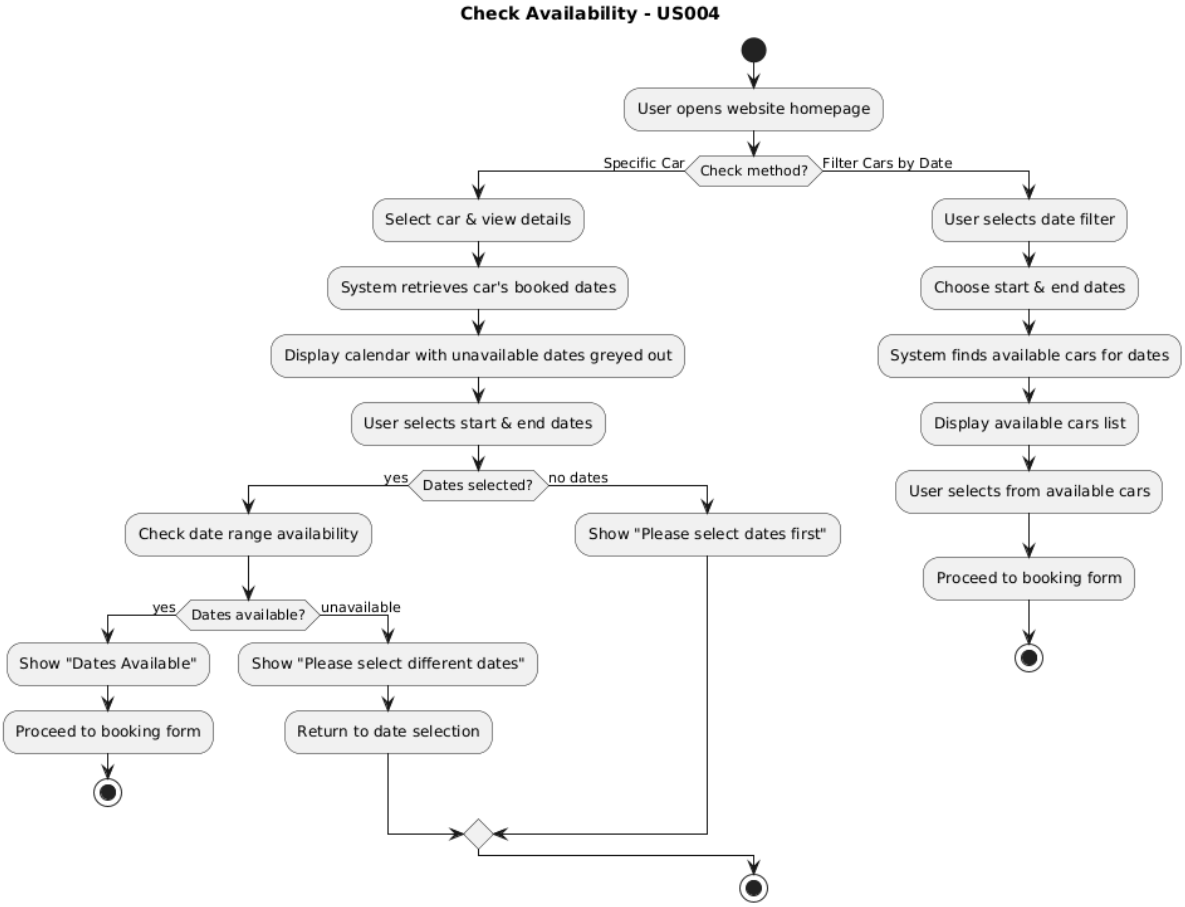

Figure 2.4.2: Activity Diagram for <Check Availability>

2.4.5 US005: User Story <Customer> Sprint 3 – Customer Module

Table 2.1 depicts User Story Description for <Register Account> use case. Figure 2.1.1 describes step-by-step interaction of the customer with the system to register an account based on Table 2.1. It shows the complete workflow for creating a new user account, beginning with the user entering their personal details into the registration interface. Figure 2.1.2 shows the sequential steps from data entry, validation, and email uniqueness checking to account creation and confirmation, including alternative paths for invalid input and duplicate emails, ultimately concluding with a successful registration.

Table 2.5: User Story Description for < Register Account >

User story: <Register Account>
ID: US005
User Story Description **As a** customer, **I want** to register an account **So that** I can access the booking system.
Flow of events: 1. Customer clicks Register button. 2. System displays the registration form. 3. Customer enters name, email, password, and confirm password. 4. Customer submits the form. 5. System validates all inputs: Required fields not empty • Email format valid • Password meets security requirements • Password and Confirm Password match 6. System checks whether the email already exists. 7. If valid, system creates a new account in the database. 8. System displays a successful registration message. 9. Customer is redirected to the login page.
Alternative flow: **Alternative flow - Empty fields** 1. System displays “Please fill in all required fields.” 2. Returns user to the registration form. **Alternative flow - Invalid Email Format** 1. System displays “Invalid email format.” **Alternative flow - Passwords Do Not Match** 1. System displays “Passwords do not match.”

Alternative flow - Weak Password 1. System displays “Password must have at least 8 characters.” **Alternative flow - Email Already Registered** 1. System displays “This email is already registered.”
Acceptance Criteria Postcondition: • A new account is created and stored in the system. Precondition: none other conditions: none
Exception flow: The system detects one or more empty fields. System displays: “Please fill in all required fields.”

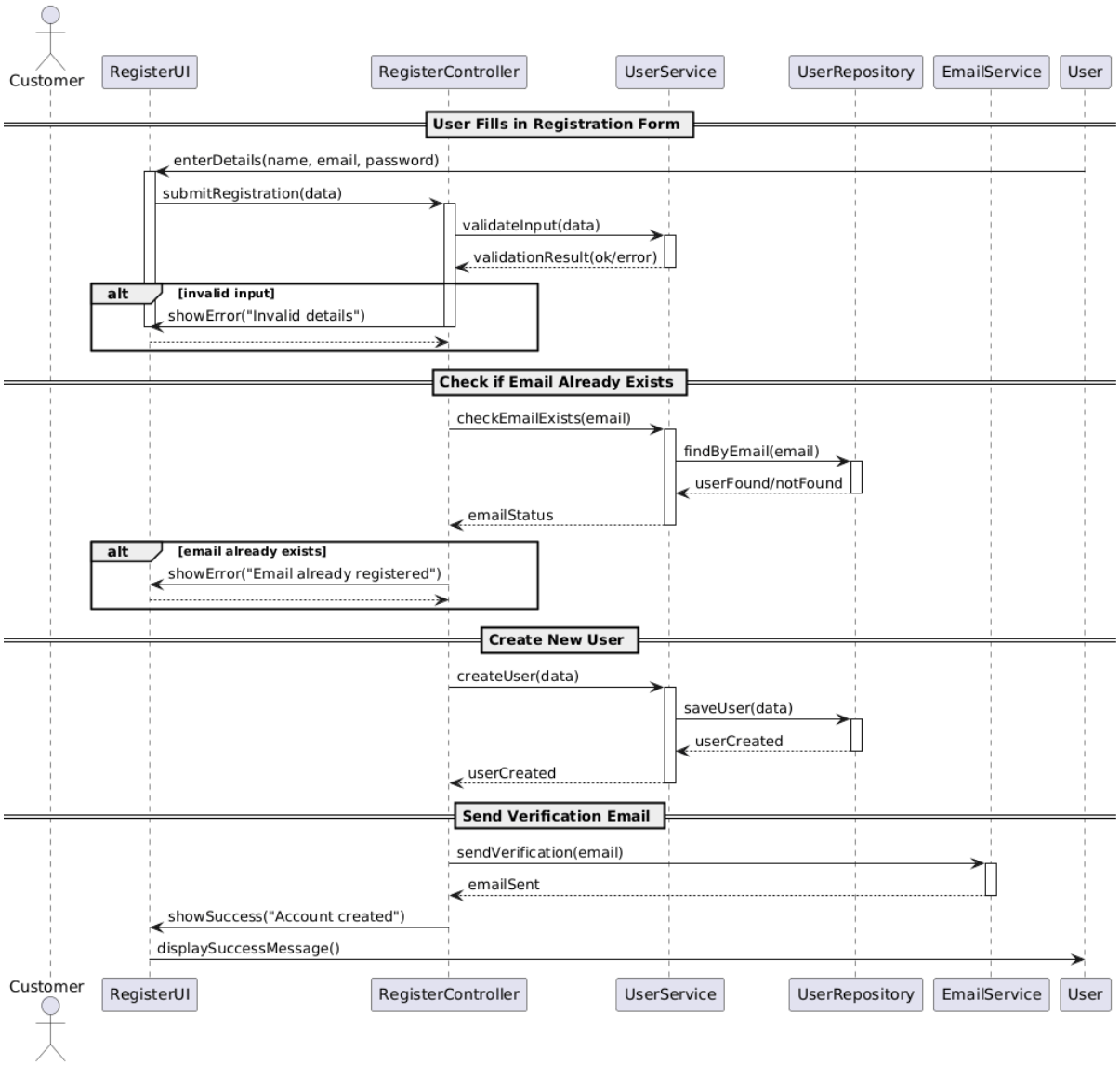

Figure 2.5.1: Sequence Diagram for <Register Account>

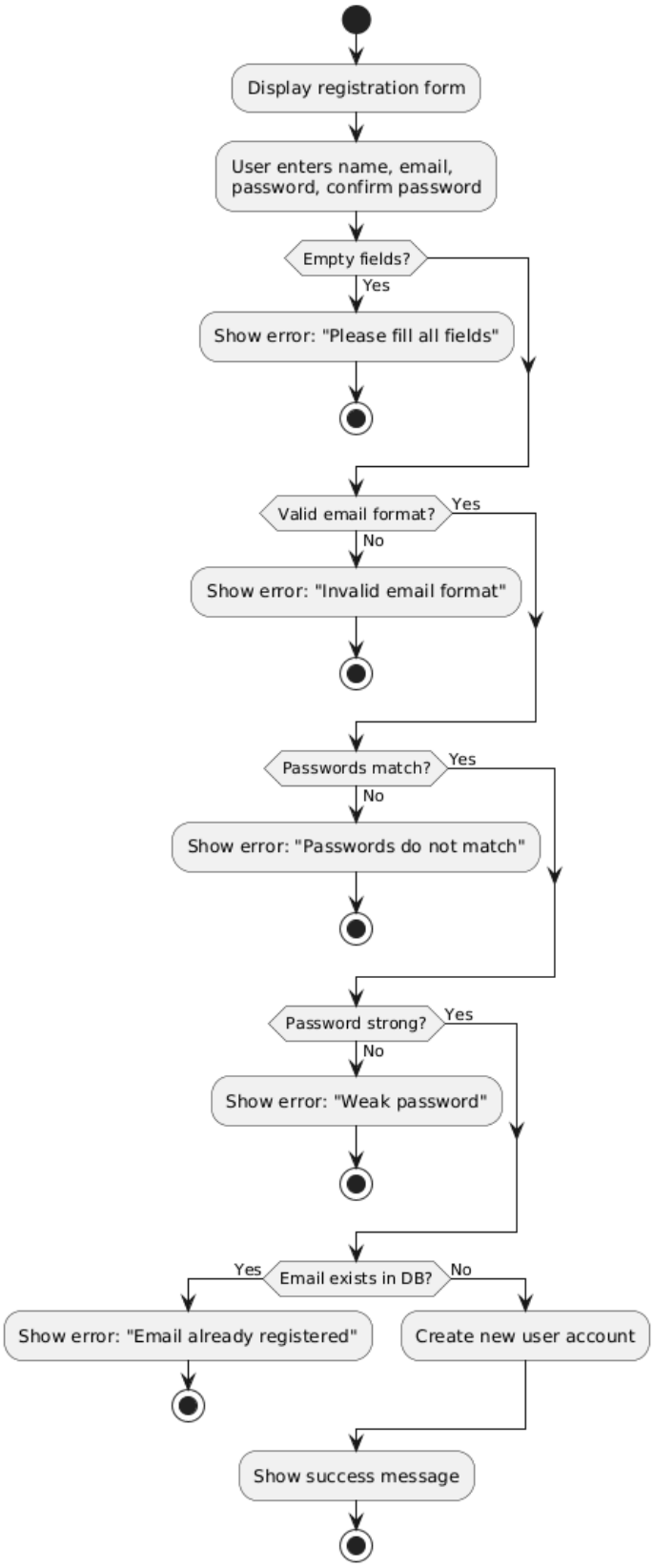

Figure 2.5.2: Activity Diagram for <Register Account>

2.4.6 US006: User Story <Customer> Sprint 3 – Customer Module

<Login Account> is a task that can be performed by a customer. Table 2.6 describes how customers can enter credentials, validate inputs, and access their dashboard. The sequence diagram in Figure 2.6.1 illustrates the interactions between the customer, login interface, authentication controller, user service, password verifier, and logger. Figure 2.6.2 shows the workflow, decision points for input validation, account verification, password check, and exception handling for incorrect credentials.

Table 2.6: User Story Description for <Login Account>

User story: <Login Account>
ID: US006
User Story Description **As a** customer, **I want** to log in to my account securely **So that** I can manage my bookings.
Flow of events: 1. Customer opens the login page. 2. System displays the login form. 3. Customer enters email and password. 4. Customer submits the login form. 5. System validates that fields are not empty. 6. System validates email format. 7. System checks if the email exists in the database. 8. System verifies the password. 9. System logs the user in. 10. System redirects the user to the dashboard.
Alternative flow: **Alternative flow - Empty field** 1. System displays: “Please fill in all fields.” **Alternative flow - Invalid Email Format** 1. System displays: “Invalid email format.” **Alternative flow - Email Not Found** 1. System displays: “Account does not exist.” **Alternative flow - Wrong Password** 1. System displays: “Incorrect password.”

Acceptance Criteria Postcondition: • Customer is authenticated and redirected to the dashboard/homepage. Precondition: • Customer must already have a registered account. other conditions: none
Exception flow: If the system detects empty email or password. The system displays: “Please fill in all fields.”

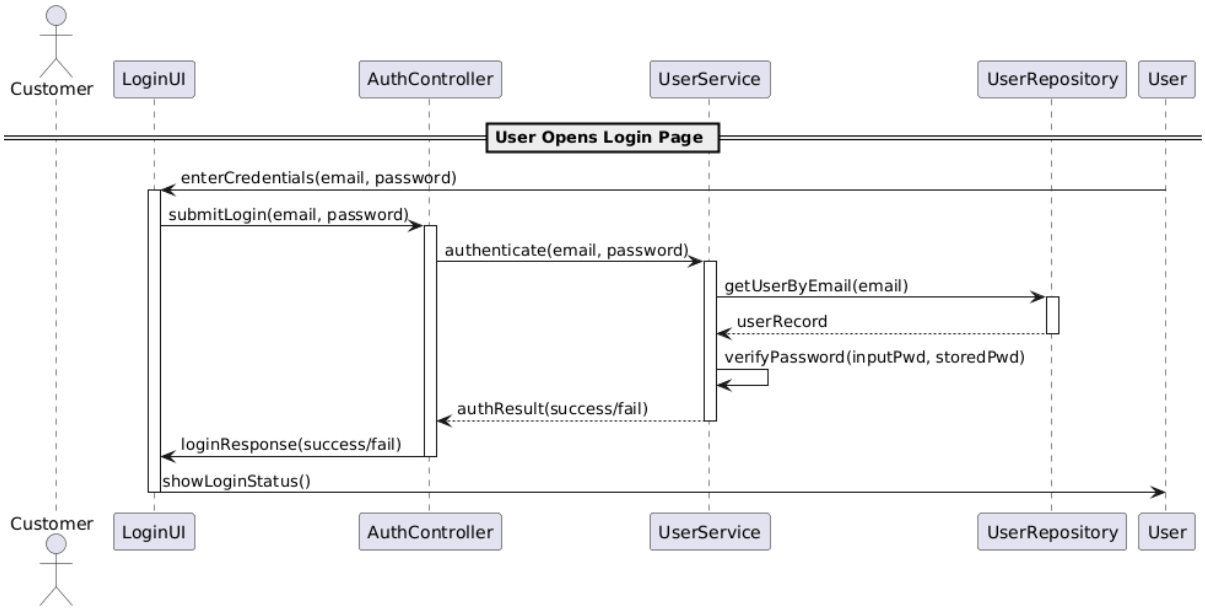

Figure 2.6.1: Sequence Diagram for <Login Account>

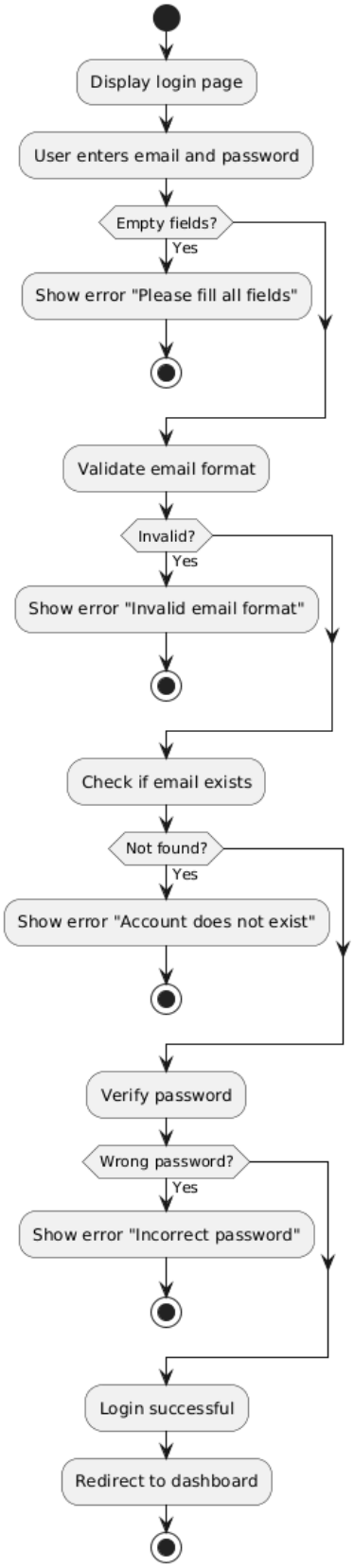

Figure 2.6.2: Activity Diagram for <Login Account>

2.4.7 US007: User Story <Customer> Sprint 3 – Customer Module

`<View Booking History>` is a task that can be performed by a customer. Table 2.7 describes how customers can request their past bookings and view details. The sequence diagram in Figure 2.7.1 illustrates the interactions between the customer, dashboard interface, booking controller, booking service, booking repository, and logger. Figure 2.7.2 shows the workflow, including decision points for session validation, retrieving booking records, and handling exceptions when no bookings are found.

Table 2.7: User Story Description for <View Booking History>

User story: <View Booking History>
ID: US007
User Story Description **As a** customer, **I want** to view my booking history **So that** I can review my past and upcoming rentals.
Flow of events: 1. Customer clicks View Booking History. 2. System receives the request. 3. System verifies that the user is authenticated. 4. System retrieves booking records from the database. 5. System displays the booking history page to the user.
Alternative flow: **Alternative flow - User does not log in** 1. System redirects to login page 2. System display “Please login to continue” **Alternative flow - No Booking Records Found** 1. System displays: “You have no bookings yet.”
Acceptance Criteria Postcondition: • System displays a list of all bookings belonging to the user. Precondition: • Customer must be logged in. • Booking records must exist in the system other conditions: none
Exception flow: If no booking records found, the system displays “You have no bookings yet.”

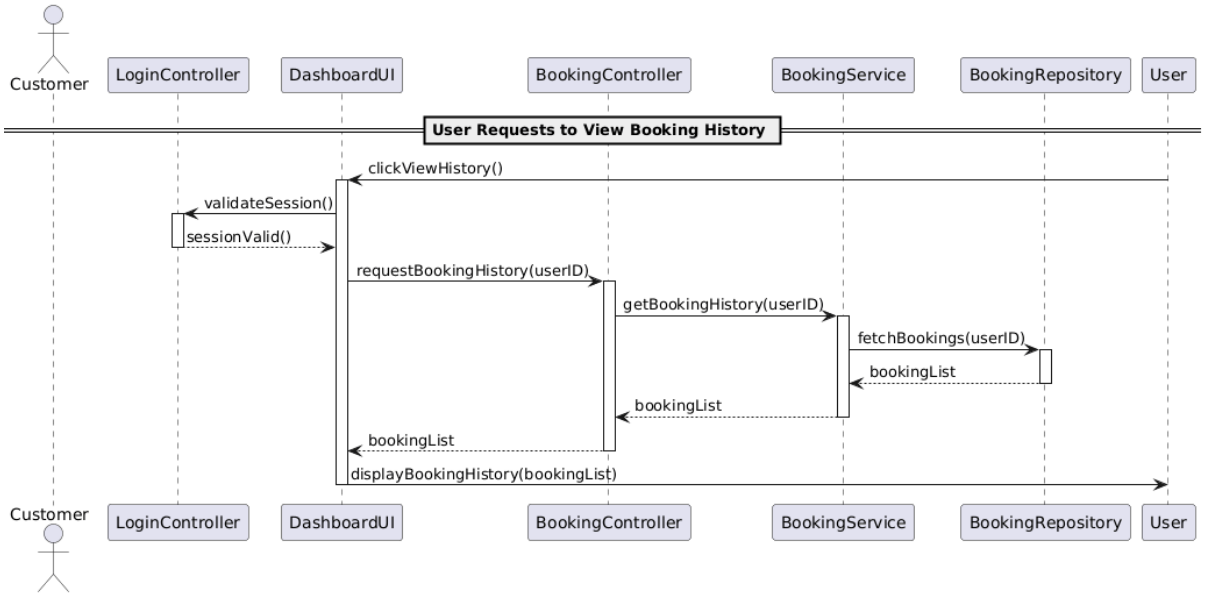

Figure 2.7.1: Sequence Diagram for <View Booking History>>

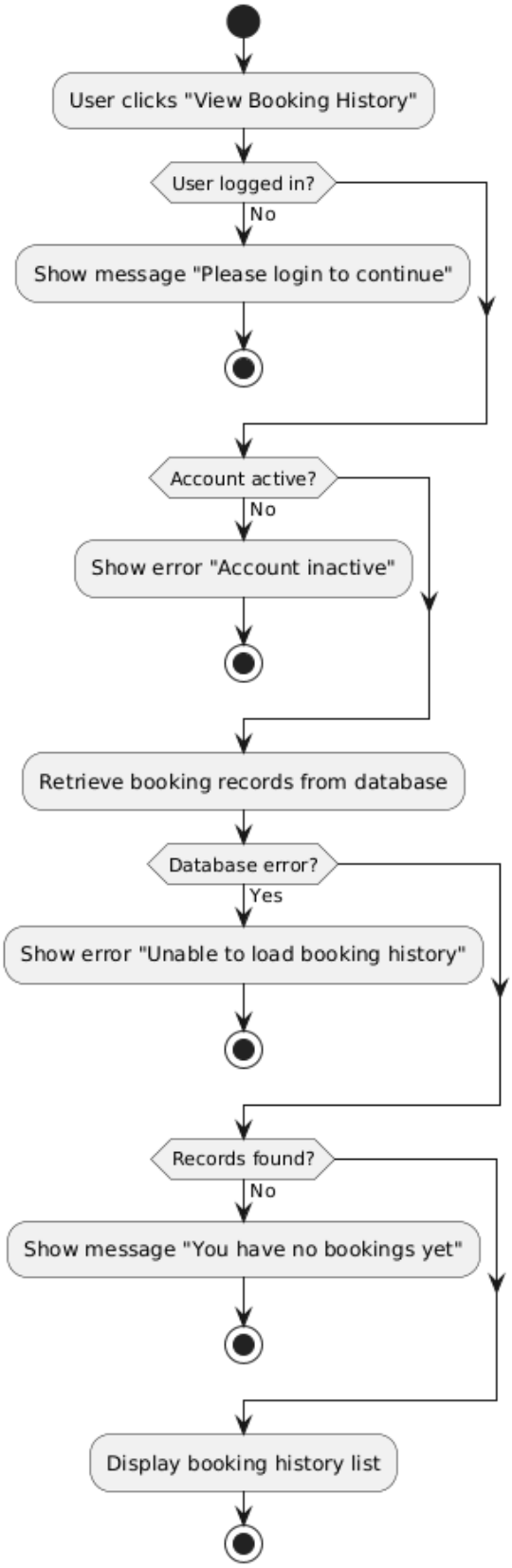

Figure 2.7.2: Activity Diagram for < View Booking History >

2.4.8 US008: User Story <Customer> Sprint 4 – Booking Design Module

Table 2.8 depicts User Story Description for <Book Car> use case. <Book Car> is a task that can be performed by a customer. Table 2.8 describes how customers can select a car, choose rental dates, check availability, make payment, and confirm the booking. The sequence diagram in Figure 2.8.1 illustrates the interactions between the customer, booking page, booking controller, booking service, car repository, booking repository, payment service, notification service, and logger. Figure 2.8.2 shows the workflow, including decision points for input validation, availability check, payment processing, booking creation, and exception handling for unavailable cars or failed payments.

Table 2.8: User Story Description for <Book Car>

User story: <Book Car>
ID: US008
User Story Description **As a** customer, **I want** to book a selected car with my preferred date and duration **So that** I can plan my travel efficiently.
Flow of events: 1. Customer selects Book Car. 2. System displays available cars and booking form. 3. Customer enters booking details 4. System checks car availability. 5. System calculates total price. 6. Customer confirms booking. 7. System creates a booking record. 8. System displays booking confirmation.
Alternative flow: **Alternative flow - User does not log in** 1. System redirects to login page 2. System display “Please login to continue” **Alternative flow - Invalid booking details** 1. System displays: “Please enter valid booking details.” **Alternative flow - Car Not Available** 1. System displays: “Selected car is not available for this time.” **Alternative flow - Payment Failure** 1. System displays: “Payment failed. Try again.”

Acceptance Criteria Postcondition: • Customer must be logged in. Precondition: • Booking confirmation is displayed to the user. other conditions: • Car is available
Exception flow: If the date or time in the past, duration negative, or missing fields. The system displays “Please enter valid booking details.”

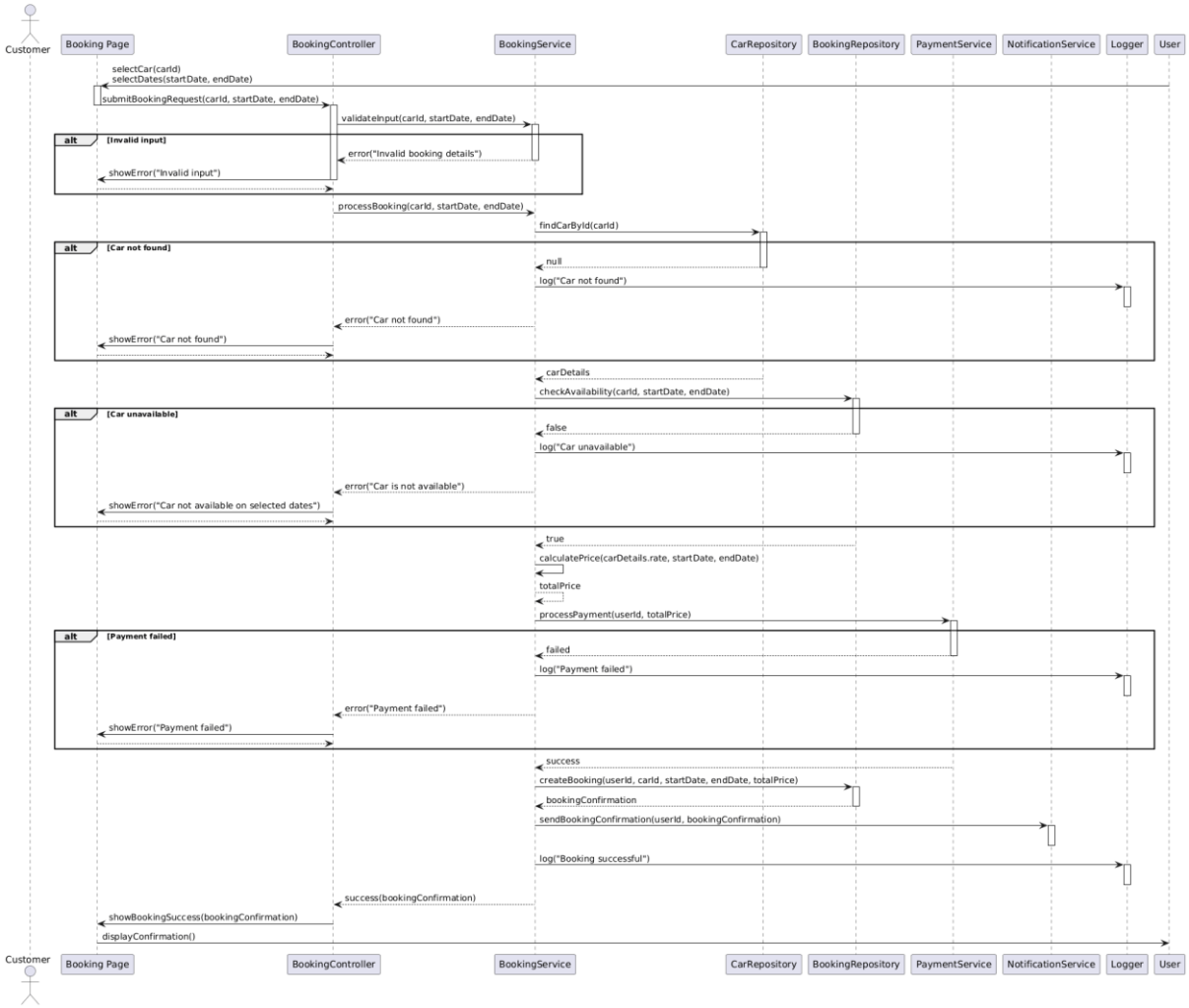

Figure 2.8.1: Sequence Diagram for <Book Car>

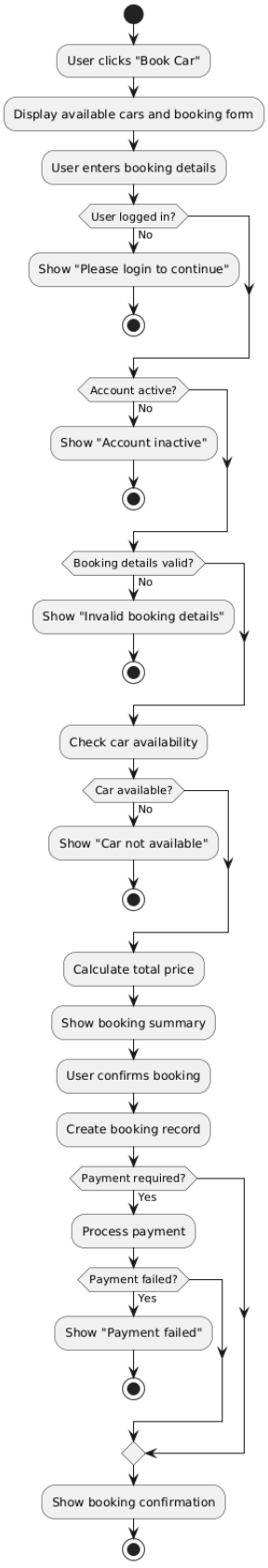

Figure 2.8.2: Activity Diagram for <Book Car>

2.4.9 US009: User Story <Staff> Sprint 3 – Customer Module

Table 2.9 depicts User Story Description for <Staff Login> use case. <Staff Login> is a task that can be performed by a staff member. Table 2.9 describes how staff can enter their credentials, validate input, and access the staff dashboard. The sequence diagram in Figure 2.9.1 illustrates the interactions between the staff, login interface, authentication controller, staff service, password verifier, and logger. Figure 2.9.2 shows the workflow, including decision points for input validation, account verification, password checking, role verification, and exception handling for incorrect credentials or suspended accounts.

Table 2.9: User Story Description for <Staff Login>

User story: <Staff Log In>
ID: US009
User Story Description **As a** staff member, **I want** log into the system **So that** I can perform booking-related functions.
Flow of events: 1. Staff opens the Staff Login page. 2. Staff enters staff ID / email and password. 3. System validates the input format. 4. System checks if staff account exists. 5. System verifies password. 6. System confirms staff role and permissions. 7. Login successful . System redirect to staff dashboard.
Alternative flow: **Alternative flow- Empty Fields** 1. System redirects to login page 2. System display “Please fill all field to continue” **Alternative flow- Invalid Format** 1. System redirects to login page 2.System will display “Invalid staff ID or email format” **Alternative flow- Wrong Password** 1. System redirects to login page 2. System display “Incorrect password.”
Acceptance Criteria Postcondition:

• Redirected to dashboard Precondition: • Already registered staff account other conditions: none
Exception flow: If the fields are empty. The system will redirect to login page and display “Please fill in all fields.”

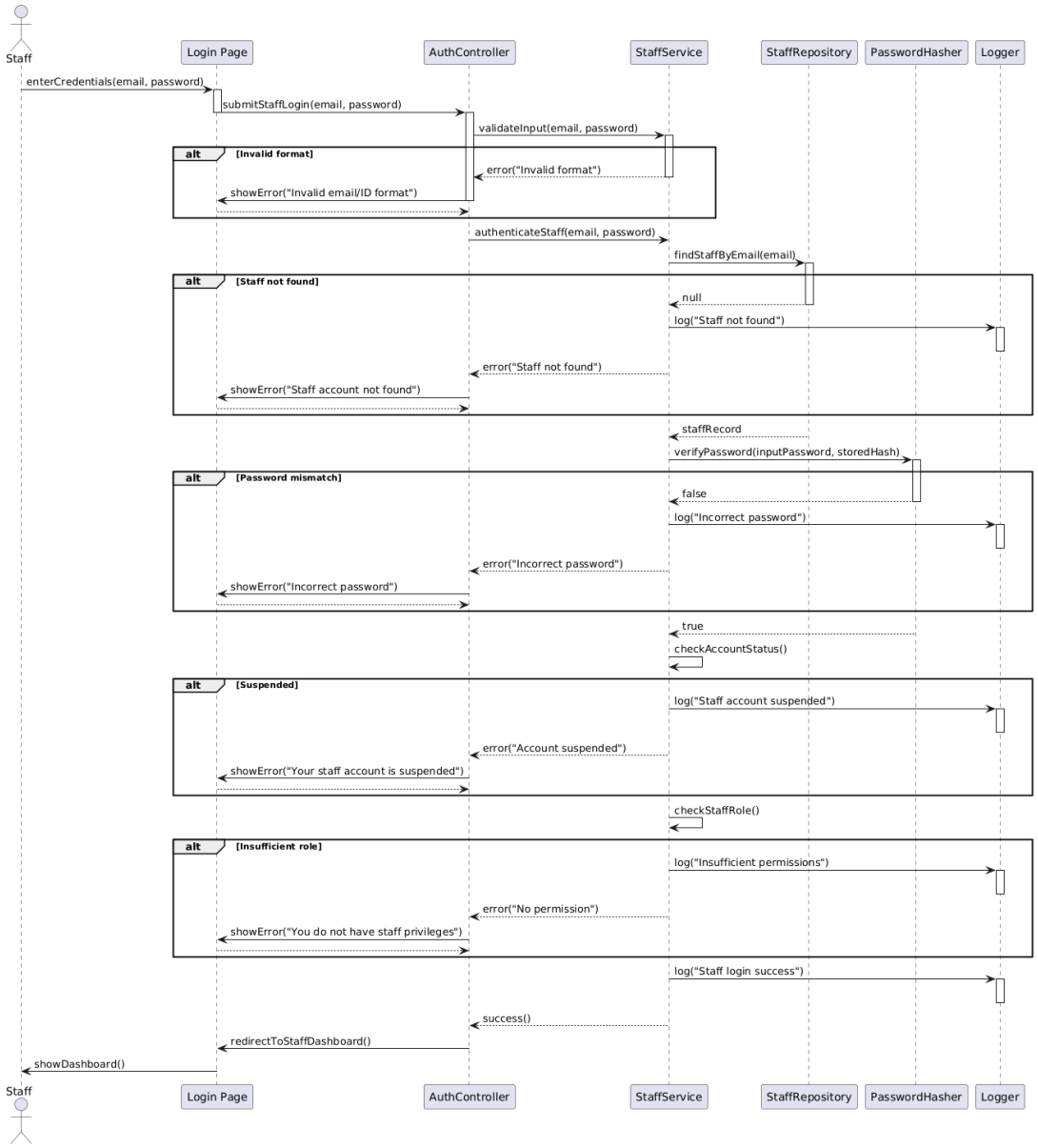

Figure 2.9.1: Sequence Diagram for <Staff Login>

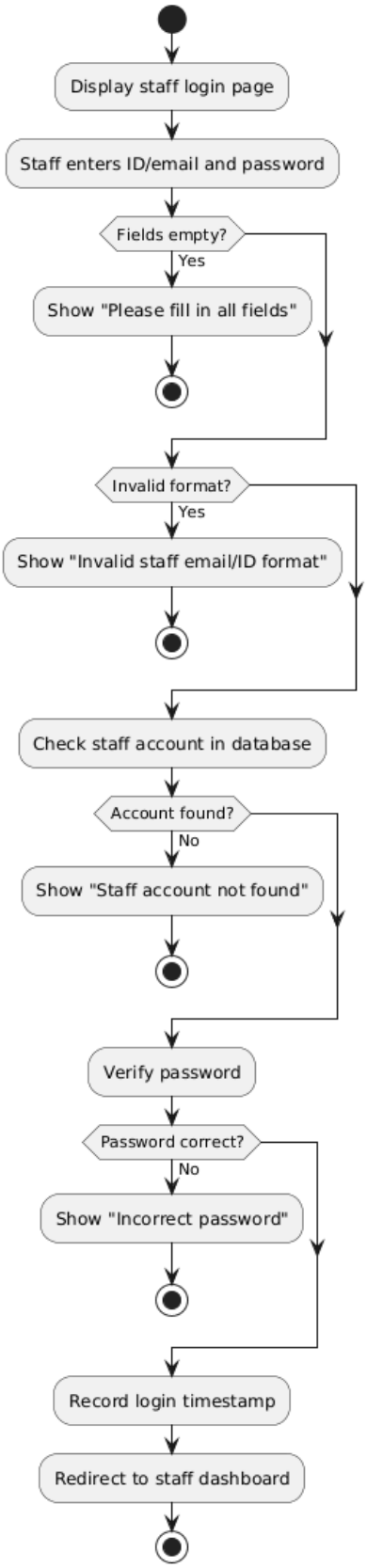

Figure 2.9.2: Activity Diagram for <Staff Login>

2.4.10 US010: User Story <Administrator> Sprint 3 – Customer Module

Table 2.10 depicts User Story Description for <Admin Login> use case. <Admin Login> is a task that can be performed by an admin, designed in the login module. Table 2.10 describes how admins can enter their credentials, validate input, and access the admin dashboard. The sequence diagram in Figure 2.10.1 illustrates the interactions between the admin, login interface, authentication controller, admin service, password verifier, and logger. Figure 2.10.2 shows the workflow, including decision points for input validation, account verification, password checking, privilege verification, and exception handling for incorrect credentials, disabled accounts, or insufficient privileges.

Table 2.10: User Story Description for <Admin Login>

User story: <Admin Login>
ID: US0010
User Story Description **As an** administrator, **I want** to log into the system **So that** I can access system-wide management functions.
Flow of events: 1. Admin opens the Admin Login page. 2. Admin enters staff ID / email and password. 3. System validates the input format. 4. System checks if staff account exists. 5. System verifies password. 6. System confirms staff role and permissions. 7. Login successful . System redirect to staff dashboard.
Alternative flow: **Alternative flow- Empty Fields** 1. System redirects to login page 2. System display “Please fill all field to continue” **Alternative flow- Invalid Format** 1. System redirects to login page 2.System will display “Invalid admin ID or email format” **Alternative flow- Wrong Password** 1. System redirects to login page 2. System display “Incorrect password.”
Acceptance Criteria Postcondition:

• Redirected to dashboard Precondition: • Already registered admin account other conditions: none
Exception flow: If the fields are empty. The system will redirect to login page and display “Please fill in all fields.”

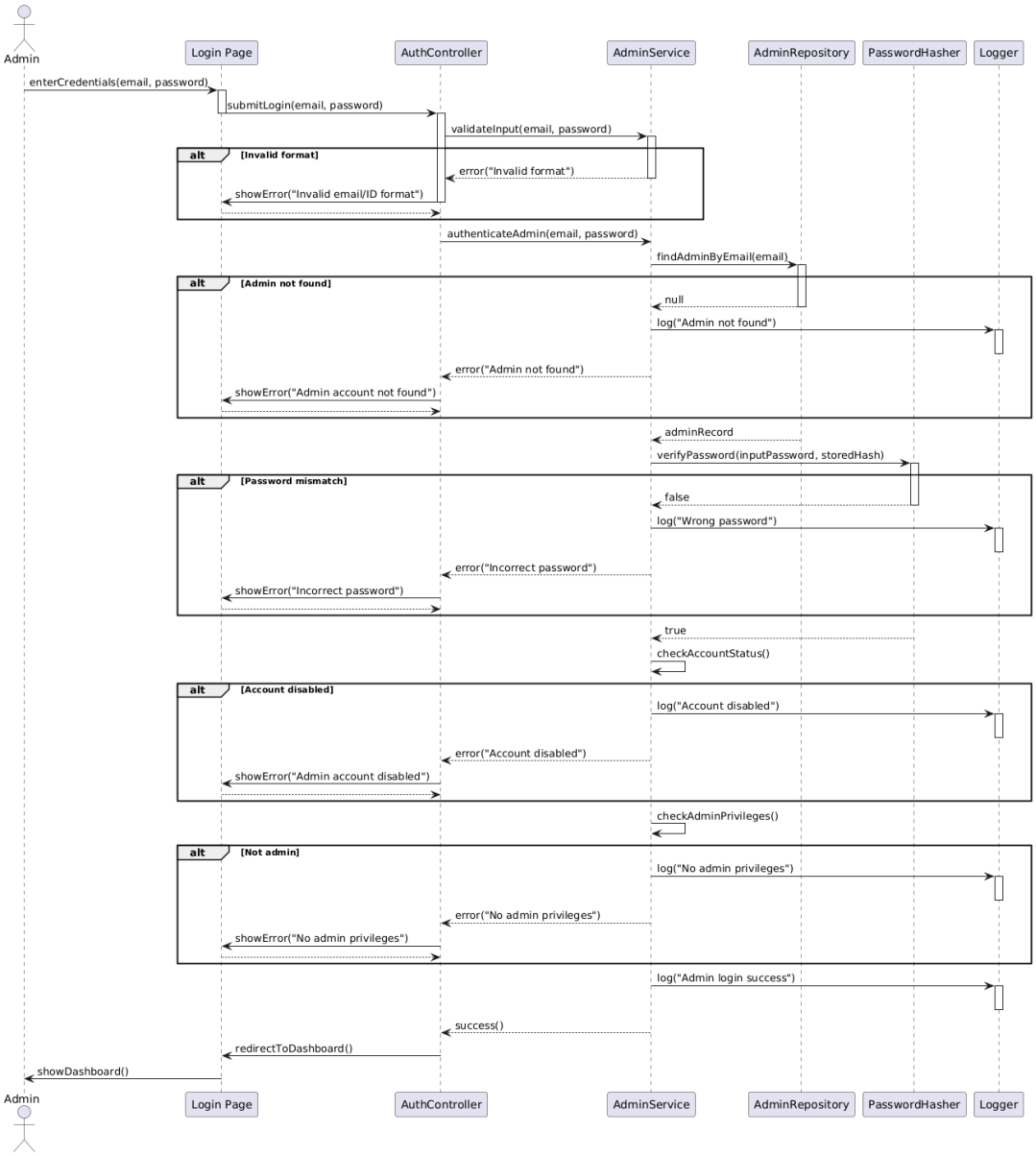

Figure 2.10.1: Sequence Diagram for <Admin Log In>

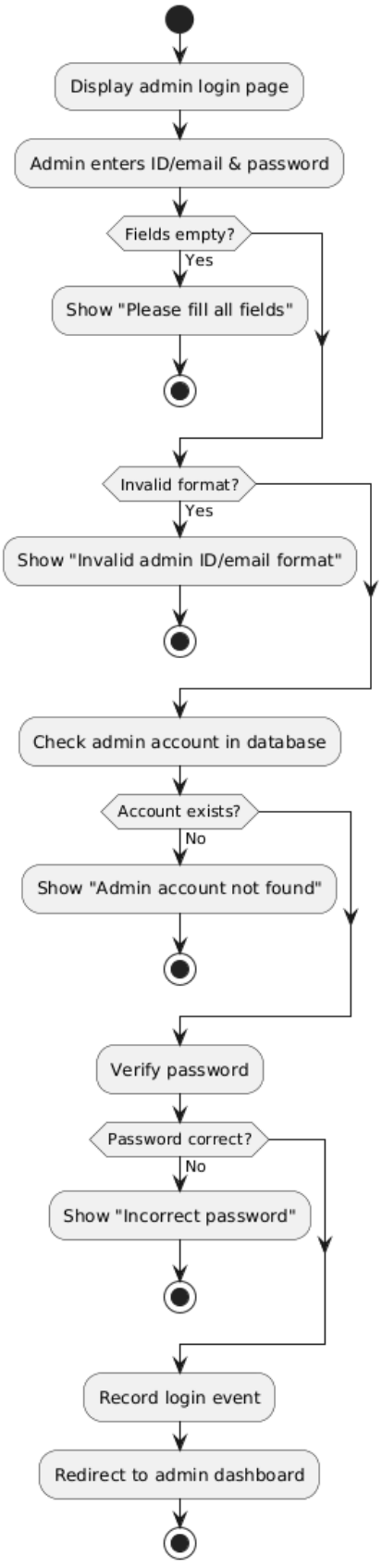

Figure 2.10.2: Activity Diagram for <Admin Log In>

2.4.11 US011: User Story <Customer> Sprint 4 – Booking Design Module

The <Cancel Booking> is in booking design module that is designed for the customer to cancel their bookings if they change their mind. Table 2.11 describes how a customer can withdraw an upcoming car booking. Figure 2.11.1 is the sequence diagram that describes the interaction between customer, booking manager, booking, policy checker and staff while Figure 2.11.2 highlights the decision points in determining whether a cancellation is allowed.

Table 2.11: User Story Description for <Cancel Booking>

User story: <Cancel Booking>
ID: US011
User Story Description **As a** customer, **I want** to cancel my booking if my plans change **So that** I do not occupy a vehicle I would not use and let staff process my cancellation.
Flow of events: 1. Customer logs into the system. 2. Customer opens the Booking History page. 3. System shows all bookings. 4. Customer selects a booking to cancel. 5. Customer clicks “Request Cancellation”. 6. System asks for confirmation. 7. Customer confirms cancellation. 8. System updates booking status to “Pending Cancellation Approval”. 9. System notifies staff on the cancellation request. 10. Customer waits for the staff to approve cancellation and credit refund.
Alternative flow: **Alternative flow – Too late cancellation is not allowed:** 1. System attempts cancellation near pickup time. 2. System denies cancellation request and displays policy message.
Acceptance Criteria Postcondition: • Booking cancellation is wait for staff approval. Precondition • Customer must be logged in to an account to cancel booking. • Booking must be “Active” and upcoming. Other conditions • Cancellation policy rules apply, no cancellation one day before pickup.
Exception flow: • If system error during cancellation, the system displays an error message • If network or database failure during cancellation, the system displays an error message for failure submission of cancellation request.

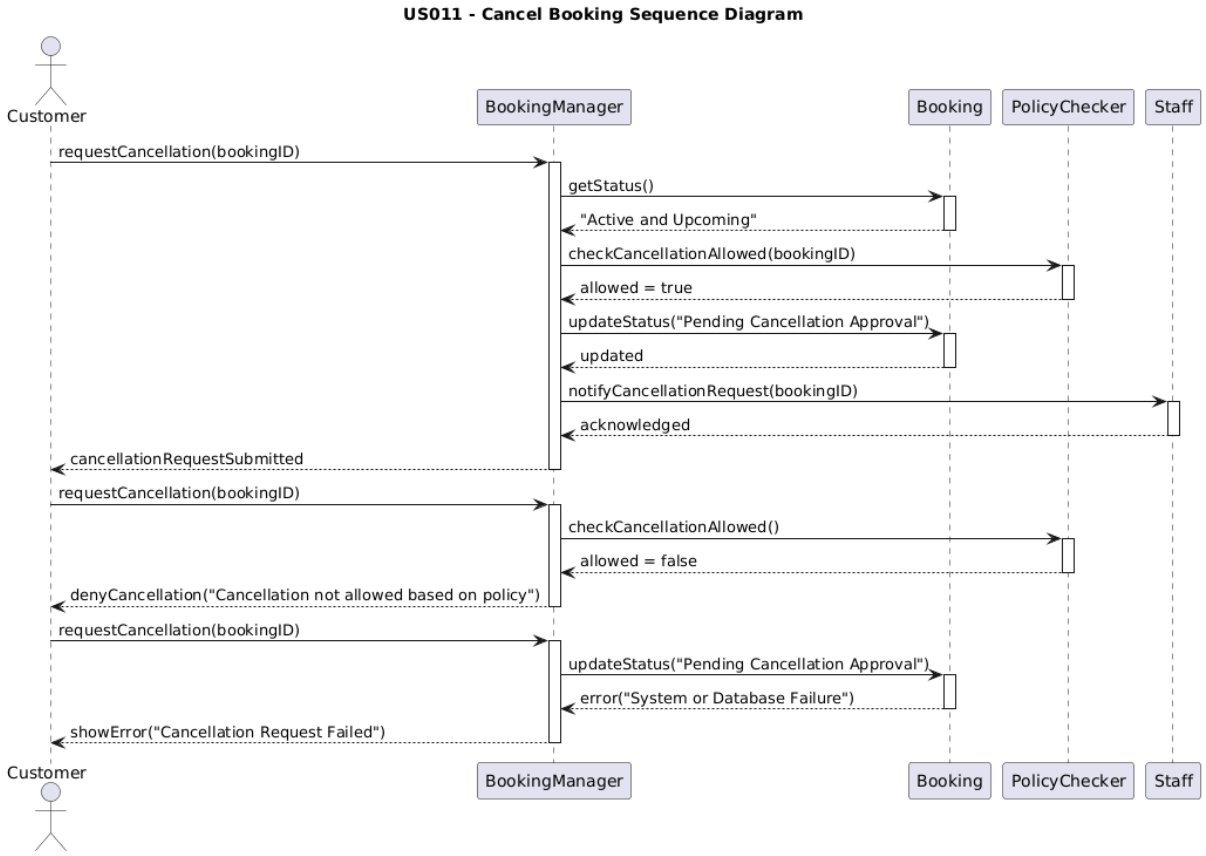

Figure 2.11.1: Sequence Diagram for <Cancel Booking>

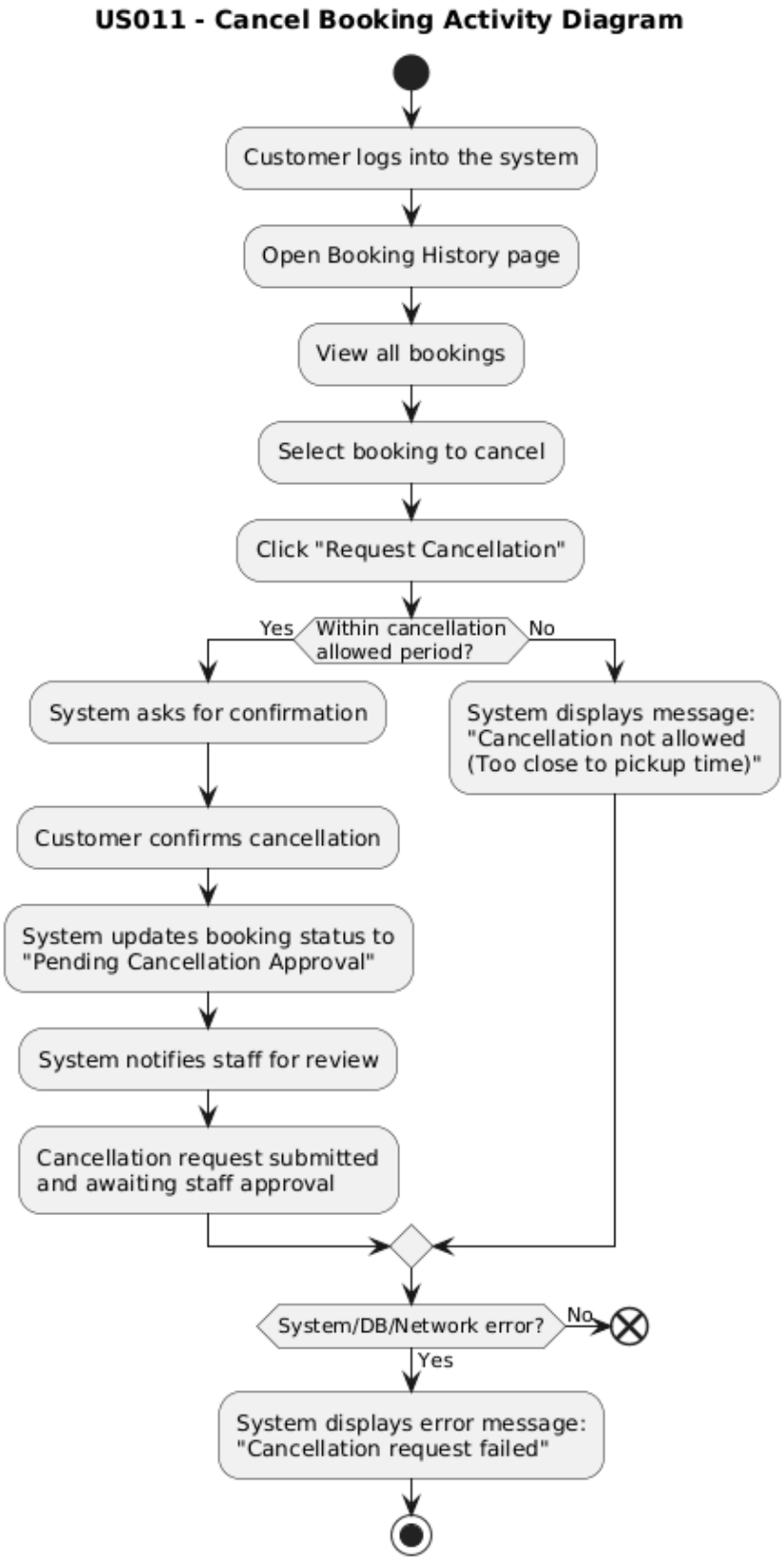

Figure 2.11.2: Activity Diagram for <Cancel Booking>

2.4.12 US012: User Story <Administrator> Sprint 5 – Admin Module

The <Manage Car> is an action can be done by admin when using Hasta Travel Booking System which is designed in admin module. Table 2.12 shows that <manage car> allows the administration to add, update and delete vehicle record. The sequence diagram in Figure 2.12.1 demonstrates the interaction between admin, vehicle manager and database. Figure 2.12.2 shows the activity diagram which focuses on the flow of actions and decision for each operations.

Table 2.12: User Story Description for <Manage Car>

User story: <Manage Car>
ID: US012
User Story Description **As an** administrator**,** **I want** to add, update, and delete vehicle records**,** **So that** I can maintain an accurate list of cars available for booking.
Flow of events: 1. Admin logs into the system. 2. Admin navigates Vehicle Management page. 3. Admin chooses to add vehicle, update vehicle or delete vehicle. 4. System displays the latest vehicle lists or vehicle registration form. 5. Admin keys in or edit vehicle details. 6. Admin saves the data. 7. System keeps the changes and refreshes the vehicle list.
Alternative flow: **Alternative flow – Add Vehicle:** 1. Admin click on button “Add Vehicle”. 2. System displays the vehicle registration form 3. Admin enters vehicle details and click save. 4. System saves the data. 5. System inserts new vehicle record into database. **6.** System updates the vehicle data. **Alternative flow – Update Vehicle:** 7. Admin click on button “Update”. 8. Admin selects a vehicle form the list 9. System displays existing details. 10. Admin edits the information and saves them. 11. System updates the vehicle data. **Alternative flow – Delete Vehicle:** 1. Admin click on button “Delete”. 2. System request confirmation. 3. Admin confirms deletion. 4. System removes vehicle from the database.
Acceptance Criteria Postcondition

• Vehicle data base can update based on the new, edited and removed vehicle data. Precondition • Admin account must be authenticated and authorized. Other conditions • Vehicle must have unique plate number. • Database connection must be available
Exception flow: • If required field is missing, the system displays an error message. • If database error occurs, system notifies admin and cancel operation.

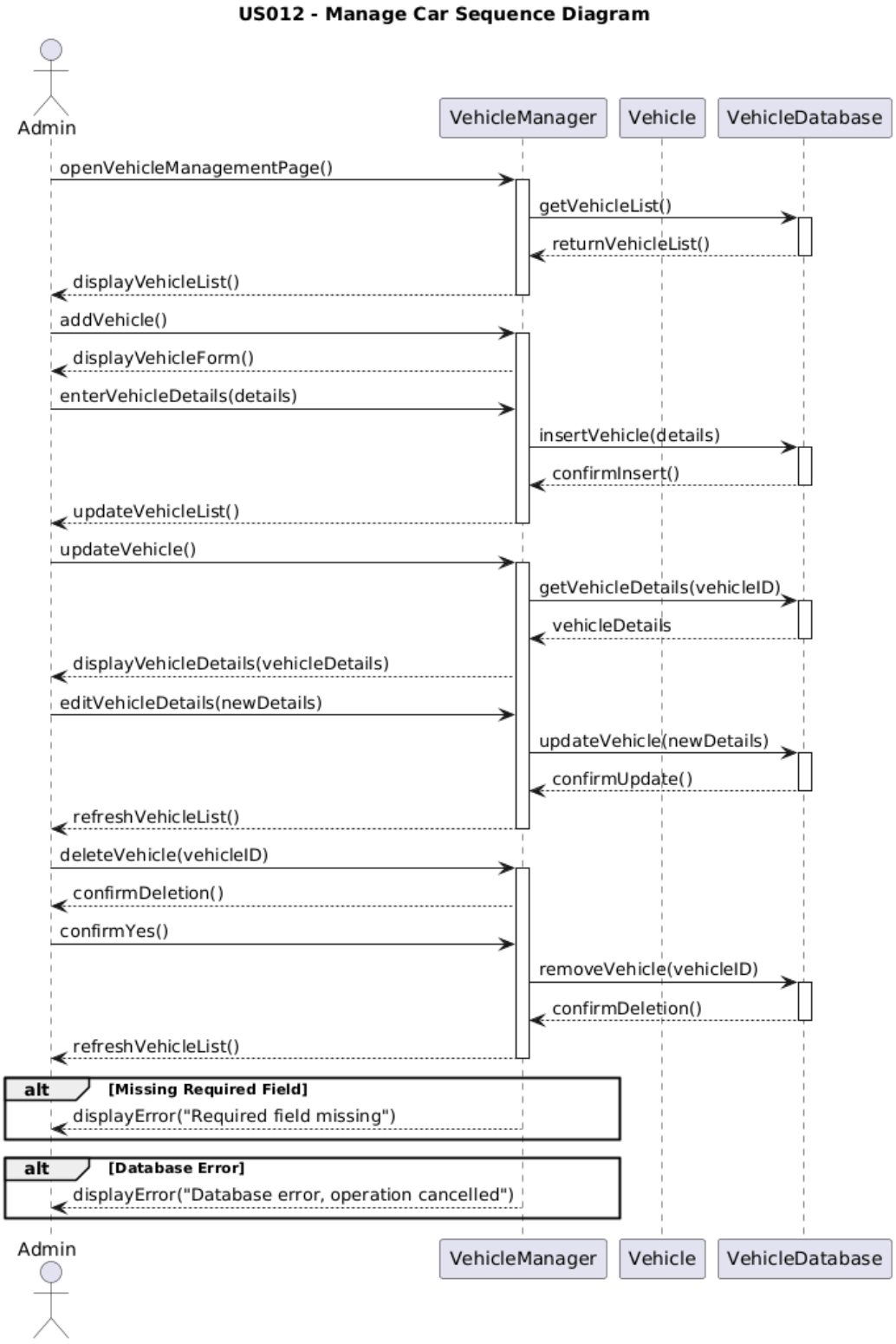

Figure 2.12.1: Sequence Diagram for <Manage Car>

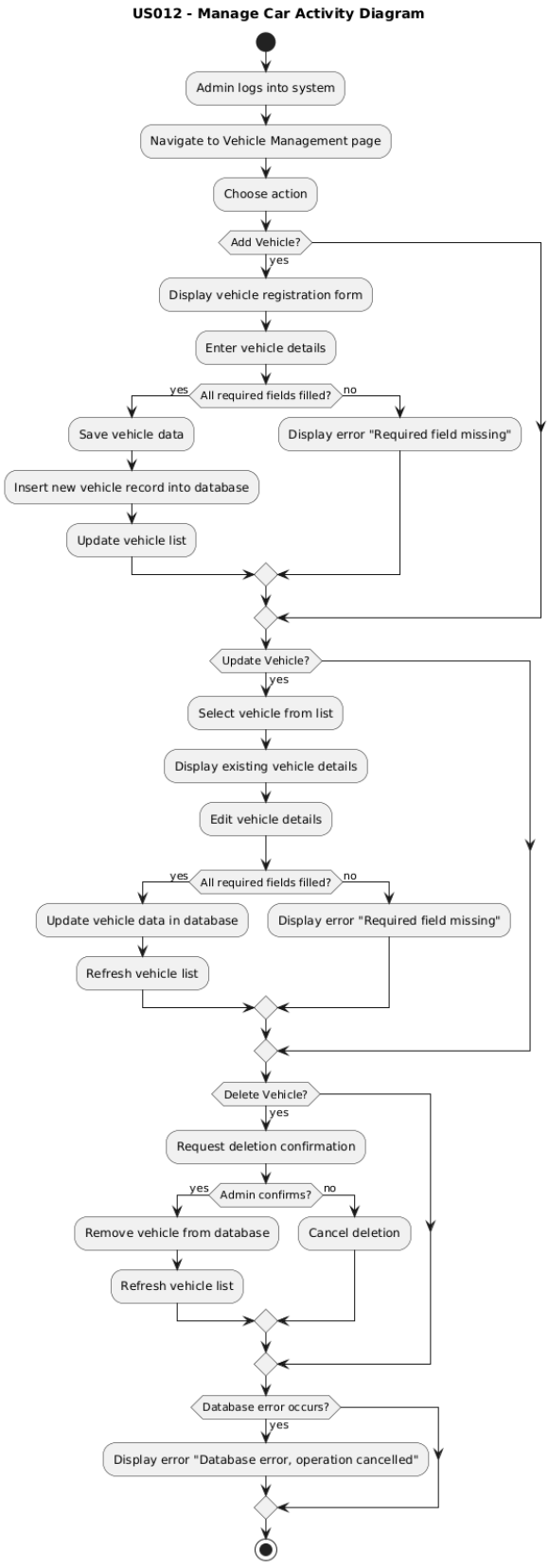

Figure 2.12.2: Activity Diagram for <Manage Car>

2.4.13 US013: User Story <Administrator> Sprint 6 - Reporting Module

The <Generate Report> is an action can be taken by admin in reporting module. In Table 2.13, it describes the <Generate Report> use case allows the administrator to generate and review rental and booking reports. The sequence diagram highlights the interaction between admin, report manager, report and database as shown in Figure 2.13.1. In Figure 2.13.2, it shows the activity diagram of flow of actions, including select report types, filter action, generating reports and export the reports.

Table 2.13: User Story Description for <Generate Report>

User story: <Generate Report>
ID: US013
User Story Description **As an** administrator, **I want** to generate rental and booking reports, **So that** I can review system performance, rental trends, and peak session.
Flow of events: 1. Admin navigates Reporting Module. 2. Admin chooses a report type such as monthly rental report, faculty rental report or car brand rental report. 3. Admin select filtering criteria. 4. Admin clicks “Generate Report”. 5. System processes the data. 6. System displays the generated report with tables and charts.
Alternative flow: **Alternative flow – Export Report:** 1. Admin clicks “Export”. 2. System exports report as PDF and Excel.
Acceptance Criteria Postcondition • Admin is able to view generated rental statistic Precondition • Booking data must exist in database • Admin must have reporting privileges.
Exception flow: • If no data found, the system display an error message • If report generation fails, the system displays error message and suggest retry.

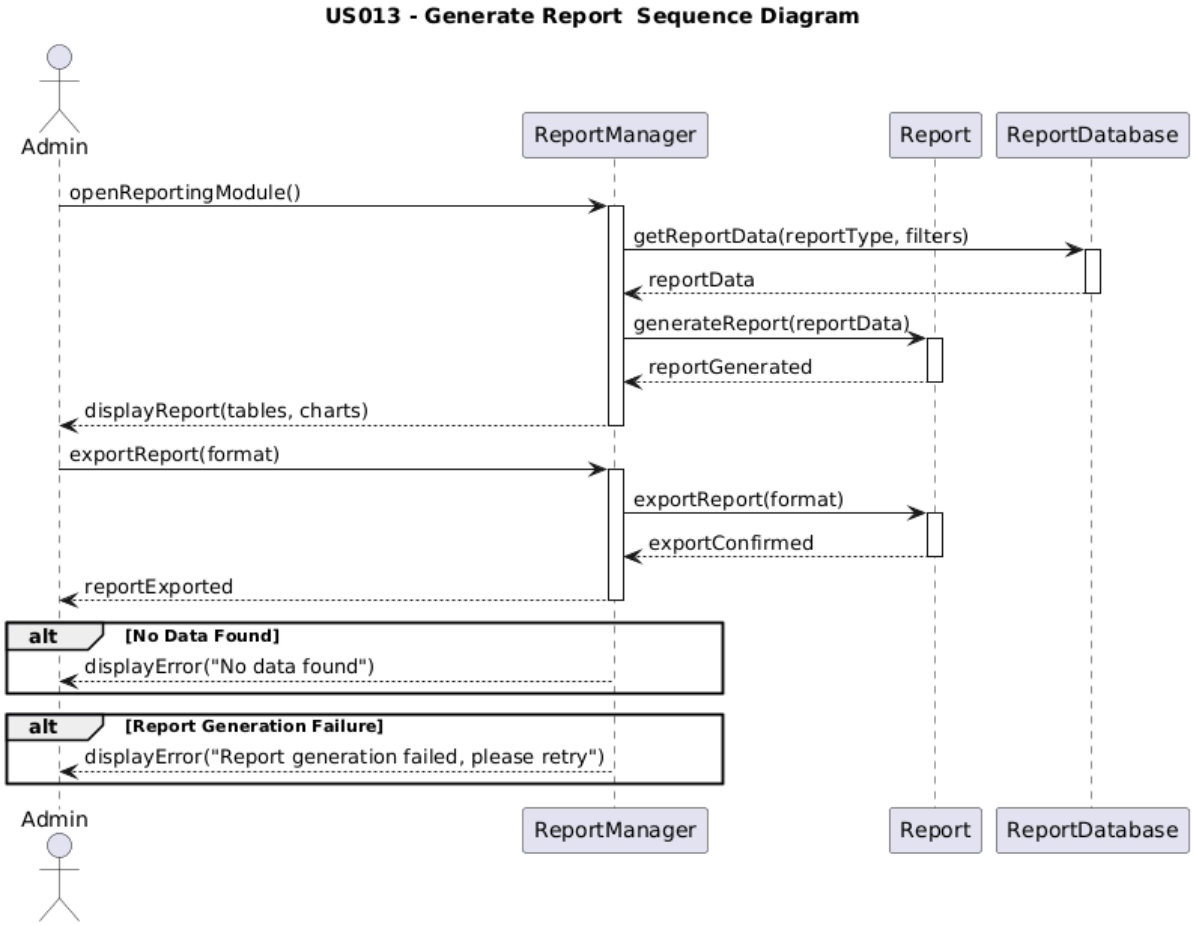

Figure 2.13.1: Sequence Diagram for <Generate Report>

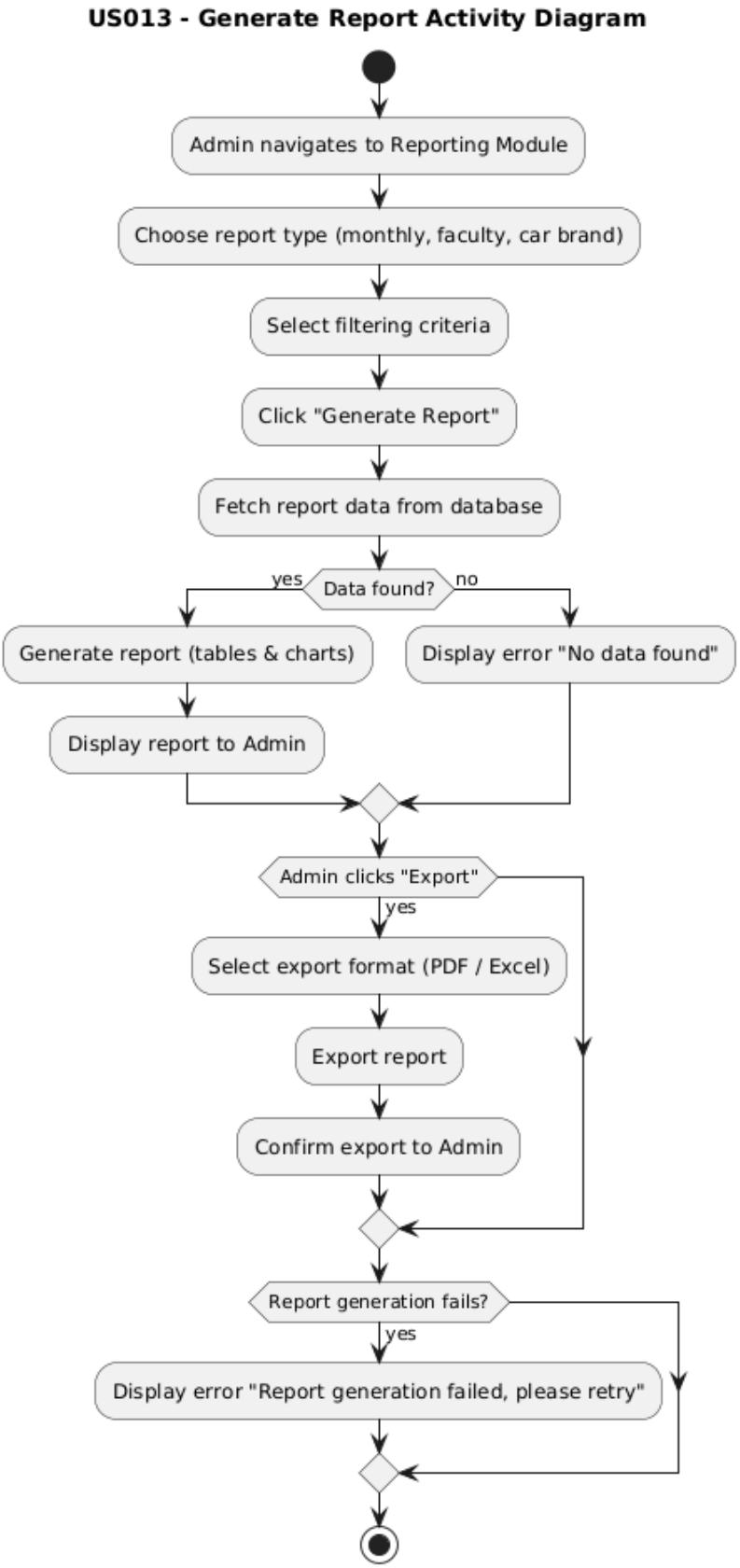

Figure 2.13.2: Activity Diagram for <Generate Report>

2.4.14 US014: User Story <Administrator> Sprint 6 –Reporting Module

The Reporting Module designed <View Report> function for administrator. Table 2.14 shows that <View Report> enables admin to access and review report generated in previous <Generate Report> user story. The sequence diagram in Figure 2.14.1 describes the interactions between admin, report reviewer, report and database. Figure 2.14.2 shows the activity diagram with flow of actions including selecting a report, load data, displaying results and apply filter.

Table 2.14: User Story Description for <View Report>

User story: <View Report>
ID: US014
User Story Description **As an** administrator, **I want** to view reports in charts and summaries **So that** I can make data-driven decisions efficiently.
Flow of events: 1. Admin opens the list of previously generated reports. 2. System displays available report categories. 3. Admin selects a report to view 4. System loads and displays charts, summaries and details.
Alternative flow: **Alternative flow – Filter in View Mode:** 1. Admin selects filter options. 2. System updates the displayed report dynamically.
Acceptance Criteria Postcondition • Admin has viewed report insights for decision making. Precondition • Reports must already be generated • Admin access required.
Exception flow: • If report fails to load, the system shows an error message.

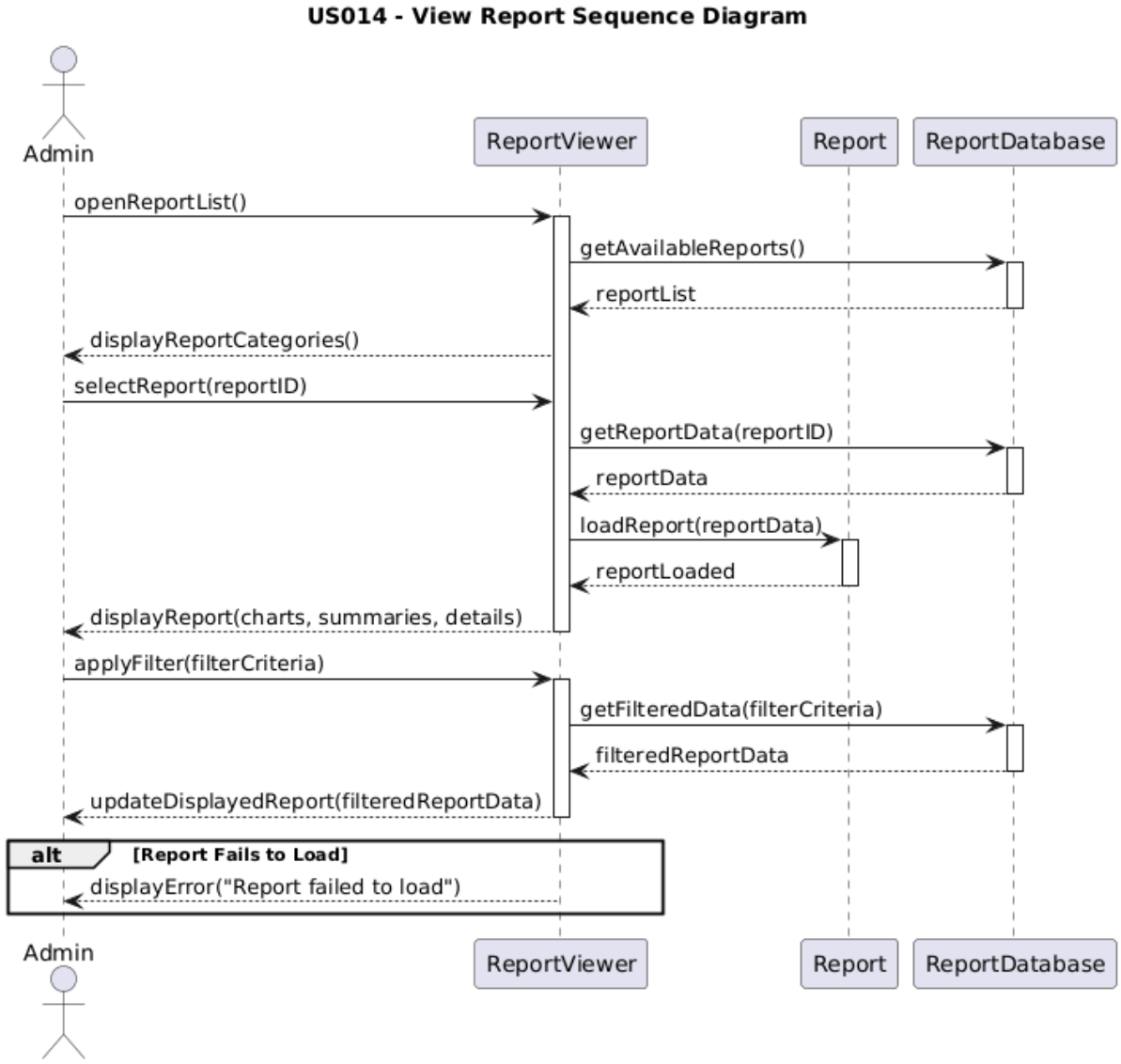

Figure 2.14.1: Sequence Diagram for <View Report>

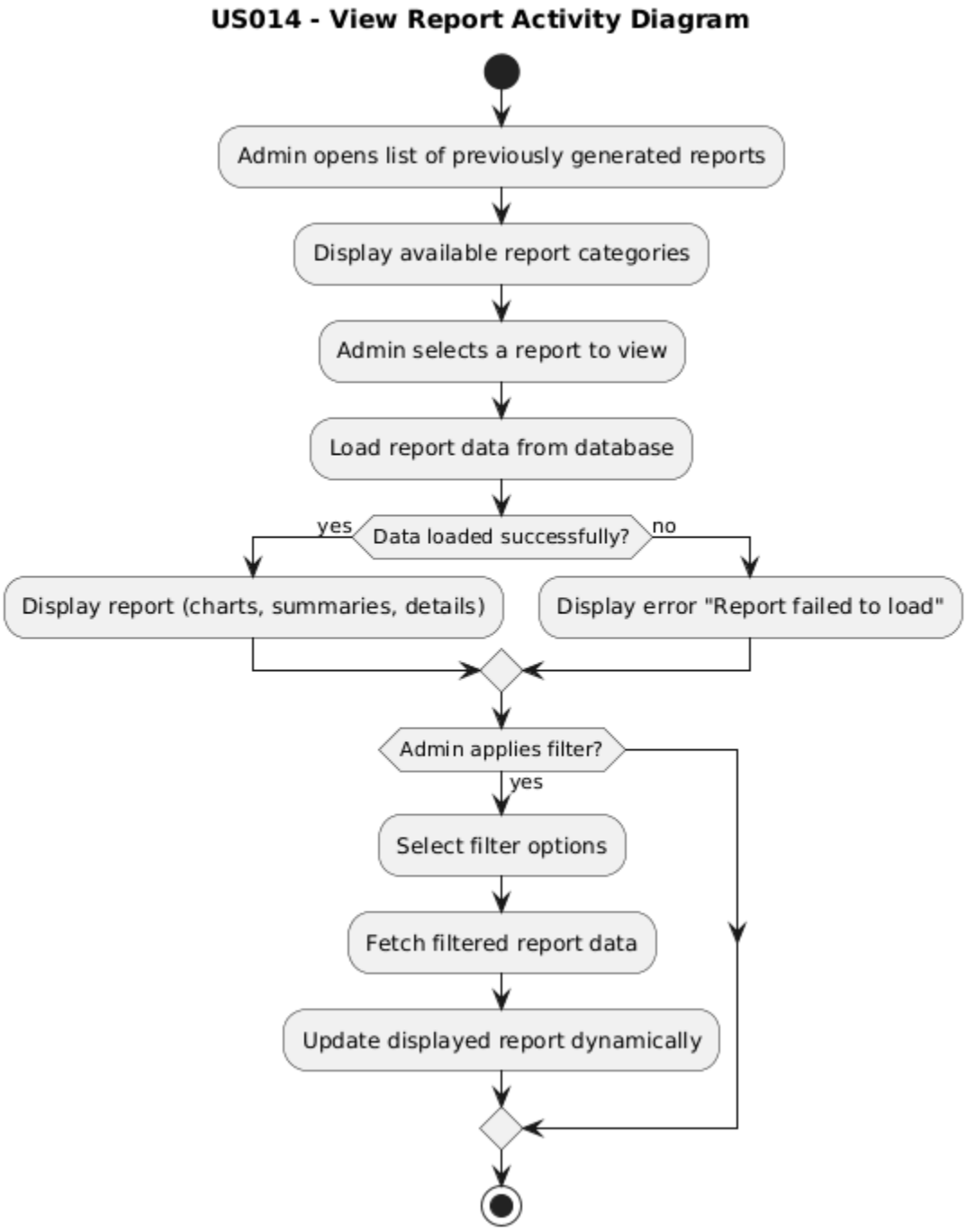

Figure 2.14.2: Activity Diagram for <View Reports>

2.4.15 US015: User Story <Staff> Sprint 7 – Booking Cancellation Module

<Process Cancellation Booking> is task that can be performed by staff designed in booking cancellation module. Table 2.15 describes how staff can review, approve or reject customer cancellation requests. The sequence diagram in Figure 2.15.1 illustrates the interactions between staff, cancellation manager, booking, refund processor, car availability and customer notifications. Figure 2.15.2 shows the workflows, decision point for approval and exception handling.

Table 2.15: User Story Description for <Process Cancellation Requests>

User story: <Process Cancellation Requests>
ID: US015
User Story Description **As a** staff member, **I want** to review and approve or reject cancellation requests submitted by customers, **So that** the booking record, refund amount, and car availability can be updated correctly.
Flow of events: 1. Staff logs into the system. 2. Staff navigates the Cancellation Request page 3. System shows all bookings with status pending cancellation approval. 4. Staff selects a cancellation request 5. System displays booking details. 6. Staff choose to approve or reject the cancellation request. If Staff Approves: 7. System updates booking status to Cancelled. 8. System checks refund eligibility and process refund if applicable. 9. System updates car availability to available. 10. System sends email notification to the customer. 11. System updates refund status. If Staff Rejects: 7. Staff enters reject reason. 8. System updates booking status to Cancellation Rejected. 9. System sends email notification to the customer.
Alternative flow: **Alternative flow –Refund:** 1. Staff approves cancellation. 2. Staff connects the customer to get the account number. 3. Staff process the refunds **Alternative flow – Not Refund:** 1. Staff approves cancellation 2. System marks refund status as “Not Eligible”.

Acceptance Criteria
• Booking is either Cancelled or Cancellation Rejected. • Database reflects updated booking status. • Customer must have submitted a cancellation request. • Staff must be authenticated. • Refund processing need customer bank account number. • Audit logs must be stored.
Exception flow: • If refund cannot be processed, system marks refund status as “Refund Pending”.

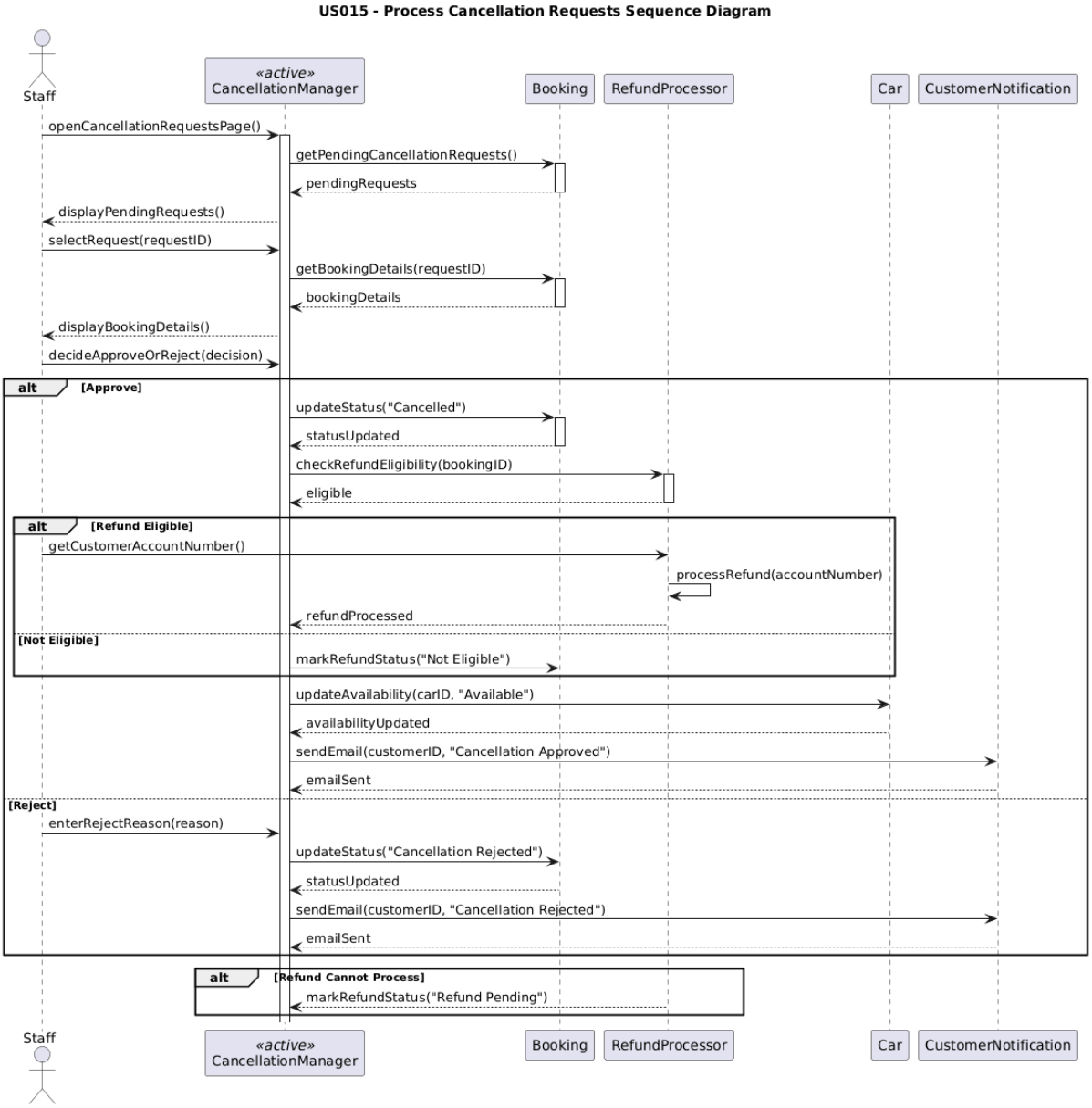

Figure 2.15.1: Sequence Diagram for <Process Cancellation Requests>

US015 – Process Cancellation Request Activity Diagram

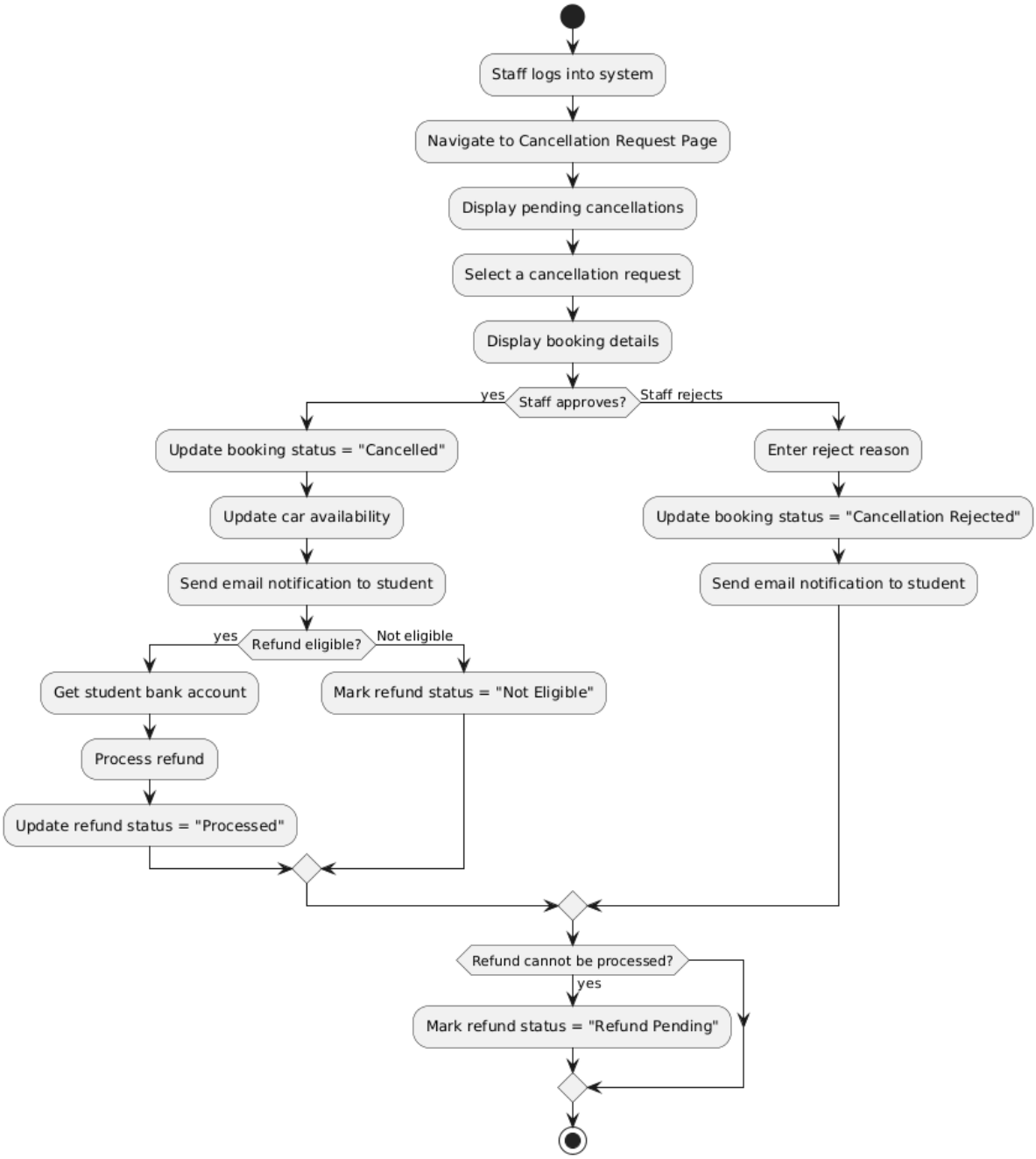

Figure 2.15.2: Activity Diagram for <Process Cancellation Requests>

2.4.16 US016: User Story <Staff> Sprint 8 – Payment Module

Figure 2.16.1 sequence diagram models the dynamic behavior of the **Staff** member verifying a payment. Figure 2.16.2: Activity Diagram for <Verify Payment> illustrates the sequential workflow for User Story US016, beginning with the staff member logging into the system and navigating to the payment verification page where the system attempts to fetch the "Awaiting Verification" list. A decision node determines the next step based on whether the list is empty; if items exist, the staff selects a booking to inspect the proof of payment image, leading to a validation decision point where the staff either approves the receipt, causing the system to update the status to "Confirmed" and notify the customer, or rejects it by inputting a reason, which updates the status to "Payment Rejected" and alerts the customer to re-upload, with both paths eventually merging to conclude the activity.

Table 2.16: User Story Description for <Verify Payment>

User story: <Verify Payment>
ID: US016
User Story Description **As a** staff member, **I want** to verify incoming payments **So that** bookings are confirmed only when payment is valid
Flow of events: 1. Staff logs into the system. 2. Staff navigate the "Payment Verification" or "Booking" list. 3. System displays bookings with status "Awaiting Payment Verification". 4. Staff selects a booking to review. 5. System displays the uploaded payment receipt image. 6. Staff compare the receipt with the company bank records. 7. Staff clicks "Approve Payment". 8. System updates booking status to "Confirmed/Paid". 9. System sends notification to the customer.
Alternative flow: **Alternative flow -Rejection Flow** 1. Staff determines the receipt is invalid or unclear. 2. Staff clicks "Reject Payment". 3. Staff enters a reason for rejection (e.g., "Image blurry", "Wrong amount"). 4. System updates booking status to "Payment Rejected". 5. System notifies the customer to re-upload the receipt.

Acceptance Criteria

Postcondition

- Booking status is updated to "Confirmed" or "Rejected" based on staff action.

Precondition

- Customer must have uploaded a receipt

Staff must be logged in.

other conditions

- The system handles manual verification as no payment gateway is integrated.

Exception flow:

If the receipt image fails to load, the system displays "Error loading image" and allows the staff to refresh the verification page.

.

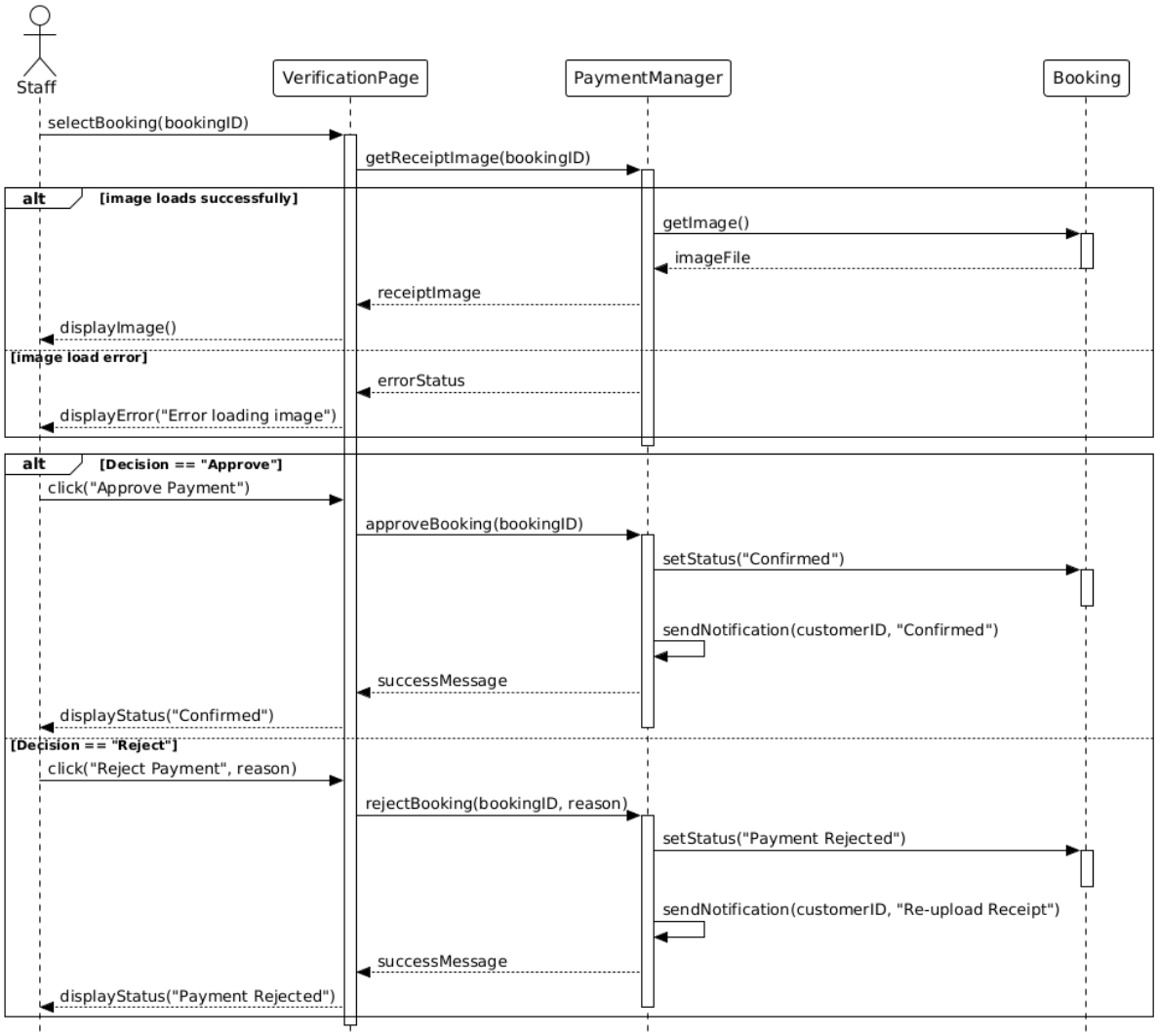

Figure 2.16.1: Sequence Diagram for <Verify Payment>

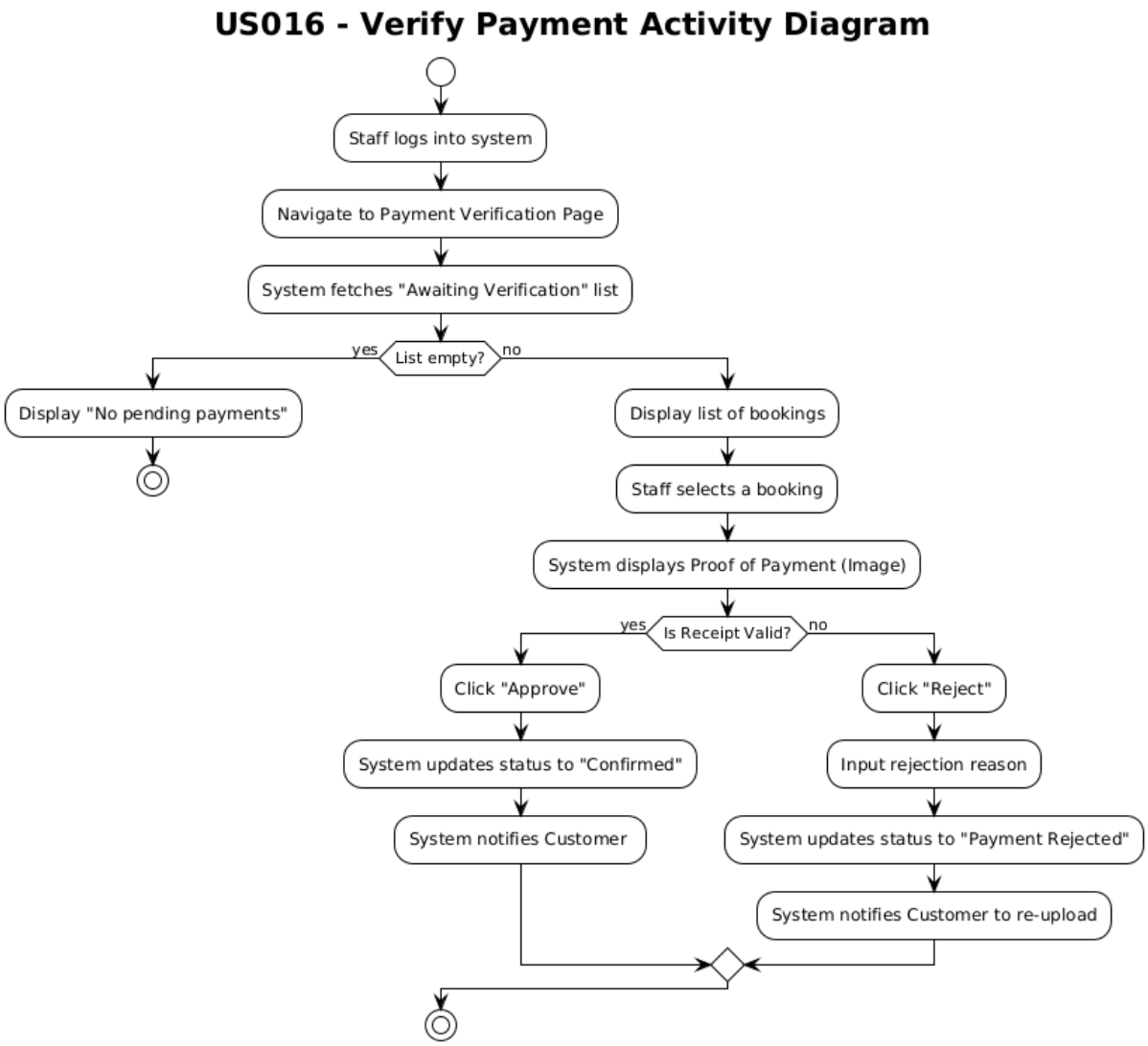

Figure 2.16.2: Activity Diagram for <Verify Payment>

2.4.17 US017: User Story <Customer> Sprint 8 – Payment Module

Figure 2.17.1 sequence diagram illustrates the interactions required for a customer to make a payment by uploading a receipt. It covers two main phases: calculating the required deposit amount (Alternative Flow) and processing the payment submission (Main Flow with Exception).

Figure 2.17.2: Activity Diagram for <Make Payment> illustrates the payment submission workflow, starting with the customer viewing the booking summary which triggers the system to calculate the required deposit based on whether the rental duration is less than 15 days (RM50) or longer (Total Rental Price). Following the display of the total amount and bank details, the customer performs a manual transfer and uploads a receipt image, leading to a validation decision where a valid format causes the system to save the receipt, update the status to "Awaiting Verification," and show a success message, while an invalid format triggers an error display and prompts a re-upload before the process concludes.

Table 2.17: User Story Description for <Make Payment>

User story: <Make Payment>
ID: US017
User Story Description **As a** customer, **I want** to make payments for my booking by uploading a receipt, **So that** my reservation is confirmed.
Flow of events: 1. Customer logs into the system. 2. Customer navigates to "My Bookings" or is redirected after booking. 3. System displays the booking summary and total amount required (Deposit + Rental) . 4. System displays Hasta Travel's bank account details. 5. Customer performs a manual bank transfer outside the system. 6. Customer clicks "Upload Receipt" and selects the proof of payment image 7. Customer clicks "Submit Payment". 8. System saves the image and updates the booking status to "Awaiting Payment Verification “
Alternative flow: **Alternative flow -Deposit Handling** 1. If the rental duration is less than 15 days, the system requests a fixed RM50 deposit . 2. If the rental duration is more than 15 days, the system requests a deposit equal to the full rental price

Acceptance Criteria
Postcondition • Receipt image is stored in the database.Booking status changes to "Awaiting Verification". Precondition • Booking must be created and in "Awaiting Payment" status. other conditions • Only manual upload is supported; no e-wallet or QR integration .
Exception flow: If the uploaded file is not a valid image format (JPG/PNG), the system displays "Invalid file format. Please upload an image."

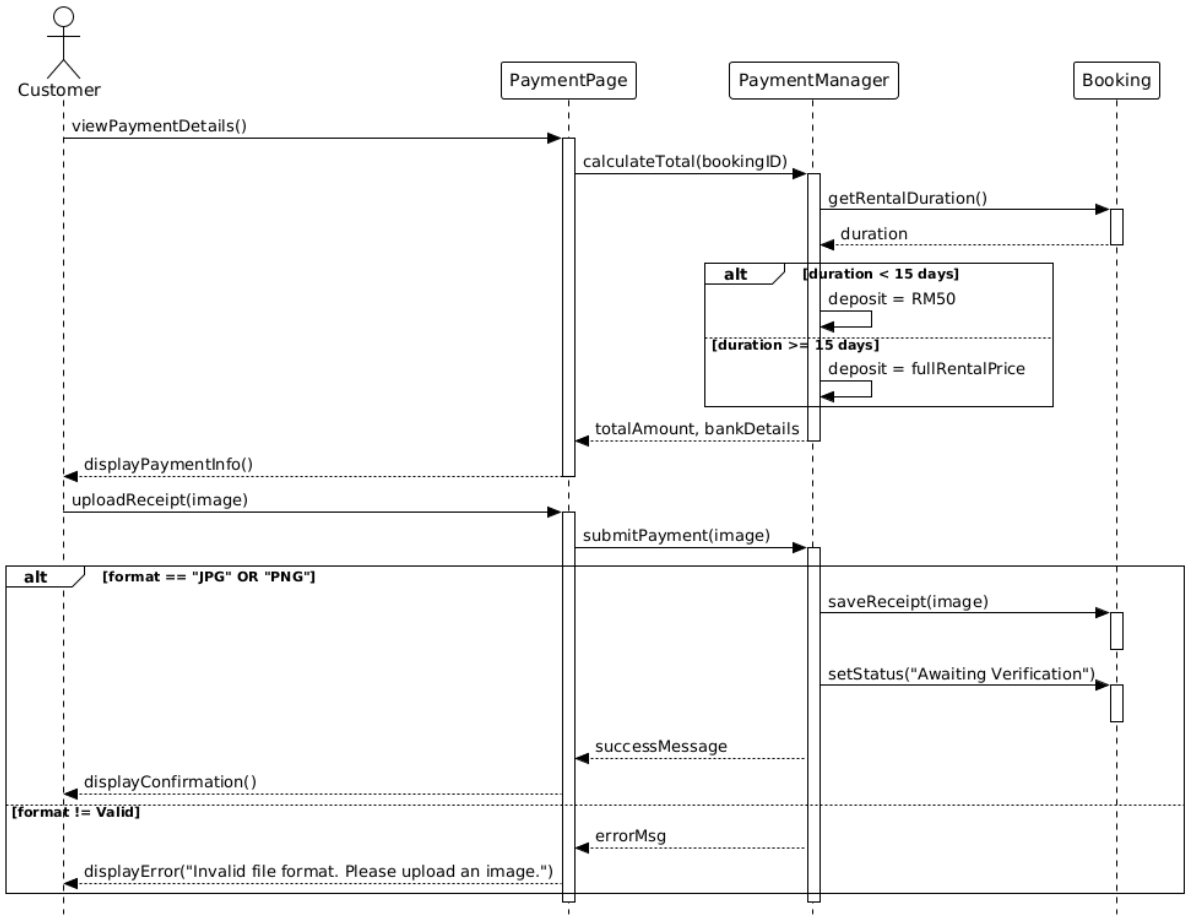

Figure 2.17.1: Sequence Diagram for <Make Payment>

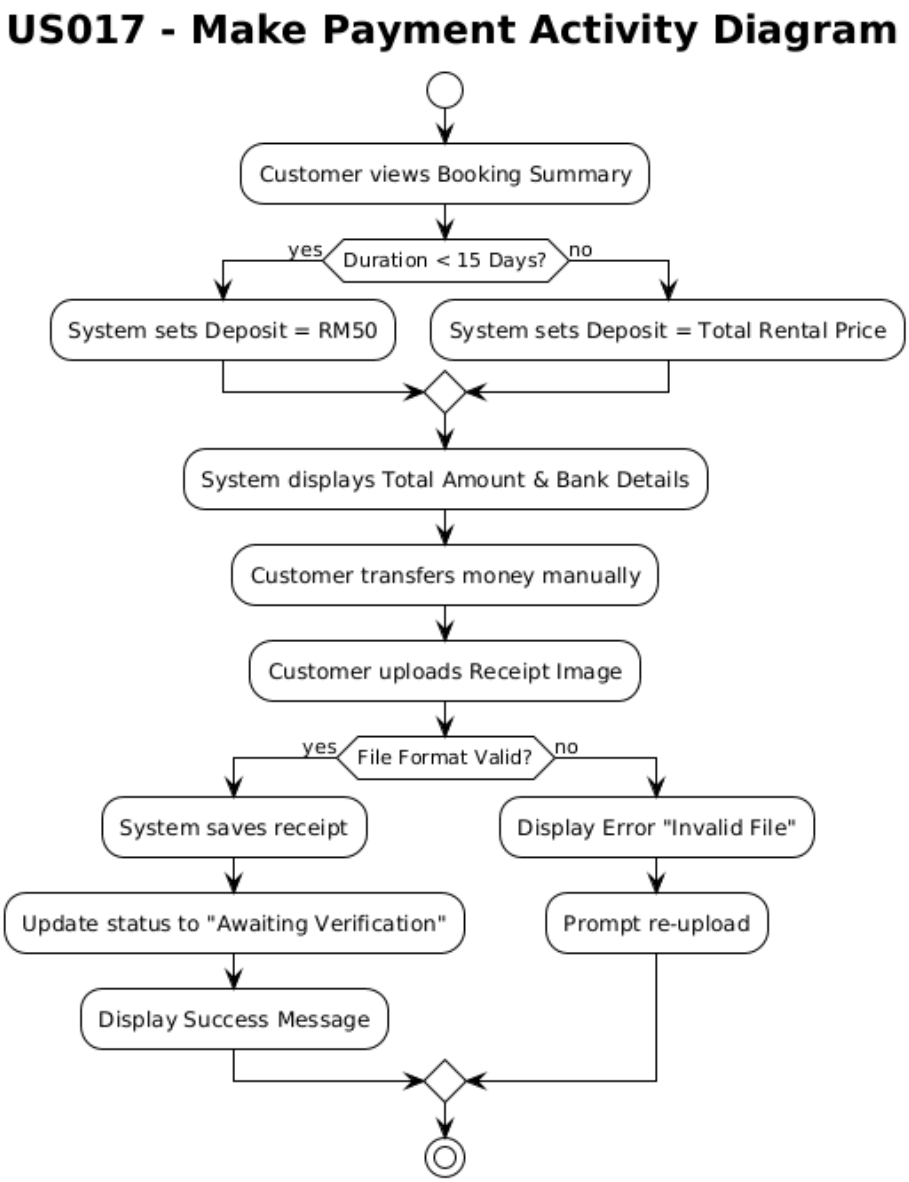

Figure 2.17.2: Activity Diagram for <Make Payment>

2.4.18 US0018: User Story <Customer> Sprint 9 – Billing Module

Figure 2.18.1 sequence diagram models the interactions between the customer(actor) and the system objects during the execution of the "Generate Invoice/Receipt" scenario. It arranges these interactions in a time sequence to show how the system responds to the user's request. Figure 2.18.2: Activity Diagram for <Generate Invoice/Receipt> illustrates the receipt retrieval workflow using swimlanes to partition actions between the customer and the System. The process initiates with the customer selecting a booking record, prompting the system to verify payment status; if unverified, the flow terminates with a status message, whereas a verified status allows the customer to request the receipt, triggering the system to generate a PDF. A subsequent decision node handles potential generation errors by displaying a failure message, while successful generation presents the receipt to the customer for viewing, with an optional path to download or print the document before the activity concludes

Table 2.18: User Story Description for Generate Invoice/Receipt

User story: <Generate Invoice/Receipt>
ID: US018
User Story Description **As a** customer, **I want** to view and download the invoice or receipt for my confirmed booking, **So that** I have an official record of the transaction and proof of my rental deposit.
Flow of events: 1. Customer logs into the system and navigates to "Booking History" or "My Bookings". 2. Customer selects a booking with the status "Confirmed" or "Payment Verified". 3. Customer clicks the "View Receipt/Invoice" button. 4. System retrieves the booking data (customer name, vehicle details, amount paid, deposit). 5. System generates and displays the receipt in PDF format 6. Customer downloads or prints the document.
Alternative flow: **Alternative flow - Email Retrieval** 1 System automatically emails the receipt to the customer immediately after payment verification (triggered by US016). 2 Customer opens their email client. 3 Customer clicks the link or attachment to view the receipt.

Acceptance Criteria

Precondition

- The booking status must be "Confirmed" and payment must be fully verified.
- The customer must be logged in.

Postcondition

- A PDF receipt is displayed or downloaded to the customer’s device.

other conditions

- Receipt must include details of the deposit paid.

Exception flow:

- If customer attempts to generate a receipt for a booking that is not yet verified, the system displays: "Receipt not available. Payment is currently being verified."
- If the PDF generation fails due to a server error, the system displays: "Unable to generate receipt. Please try again later.

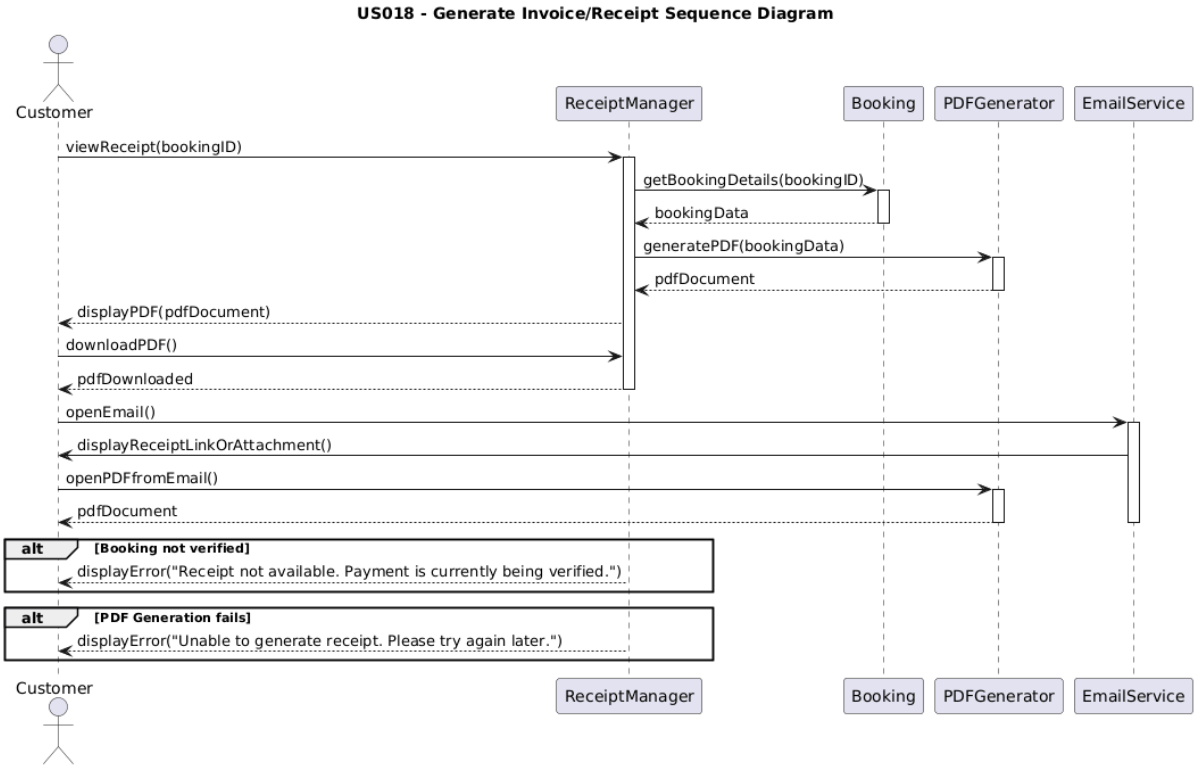

Figure 2.18.1: Sequence Diagram for <Generate Invoice/Receipt>

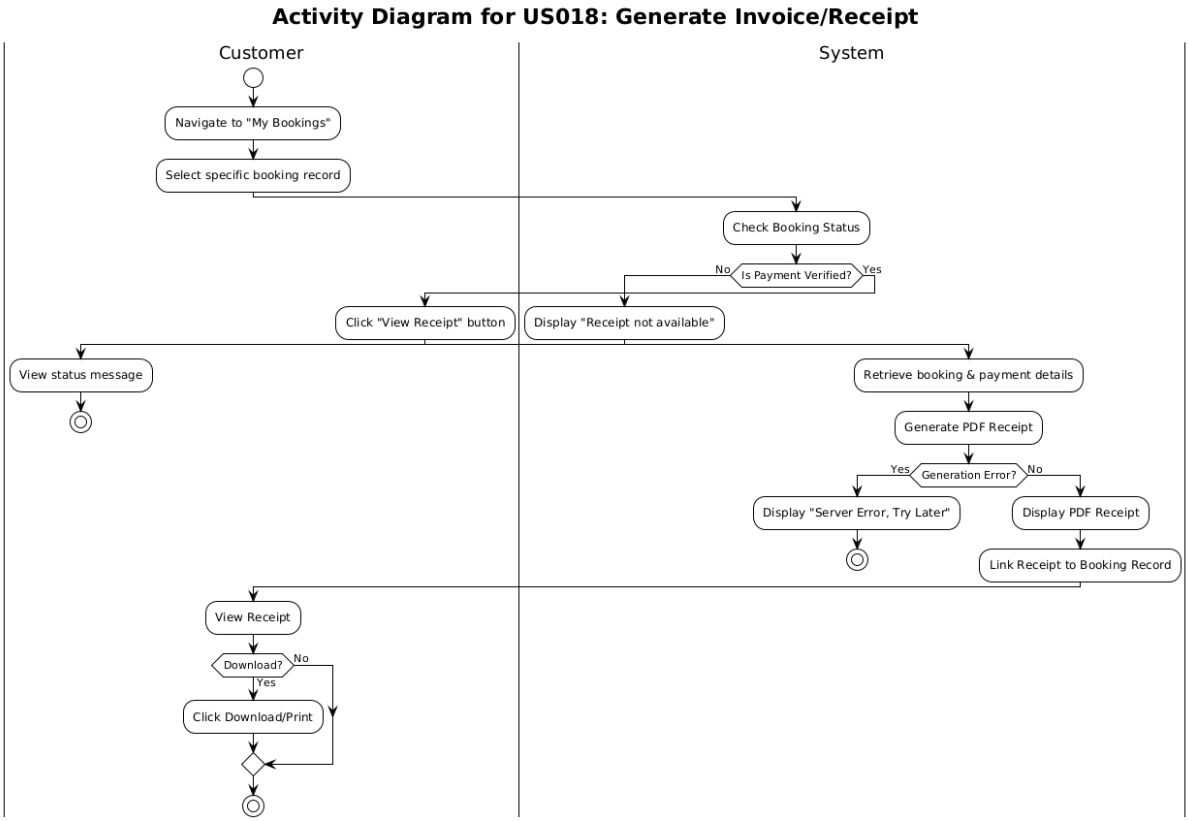

Figure 2.18.2: Activity Diagram for <Generate Invoice/Receipt>

2.4.19 US0019: User Story <Staff> Sprint 9 – Billing Module

Figure 2.19.1 sequence diagram models the dynamic behavior of the system when a Staff member initiates the invoice generation process.Figure 2.19.2: Activity Diagram for <Generate Invoice/Receipt> illustrates the automated system workflow for Table 2.19 User Story US019, which is initiated by the external event of a staff member verifying a payment (triggered from US016). The process follows a linear sequence where the system triggers invoice generation, fetches the relevant booking and payment details, and creates a PDF document; this document is then stored in the database and linked to the booking record, concluding with the system enabling the "Download Receipt" button for the customer.

Table 2.19: User Story Description for <Generate Invoice/Receipt>

User story: <Generate Invoice/Receipt>
ID: US0019
User Story Description **As a** staff member, **I want** the system to generate invoices and receipts after payment verification, **So that** customers receive official records and the accounting is accurate .
Flow of events: 1. Staff successfully verifies a payment (Triggers from US016). 2. System automatically generates a digital receipt/invoice containing booking ID, customer name, vehicle details, and amount paid. 3. System links the generated document to the specific booking record. 4. Staff (or customer) navigates to "Booking Details". 5. User clicks "View Receipt". 6. System displays the receipt in PDF format.
Alternative flow: **Alternative flow - Manual Generation** 1. Staff navigates to a completed booking. 2. Staff clicks "Generate Invoice". 3. System creates the document and makes it available for download.
Acceptance Criteria Postcondition: • A digital receipt is accessible to the customer and staff. Precondition: • Payment must be verified and status must be "Confirmed" . Other conditions: • Receipt must include details of the deposit paid.
Exception flow: If the PDF generation fails due to server error, the system displays "Unable to generate receipt. Please try again later."

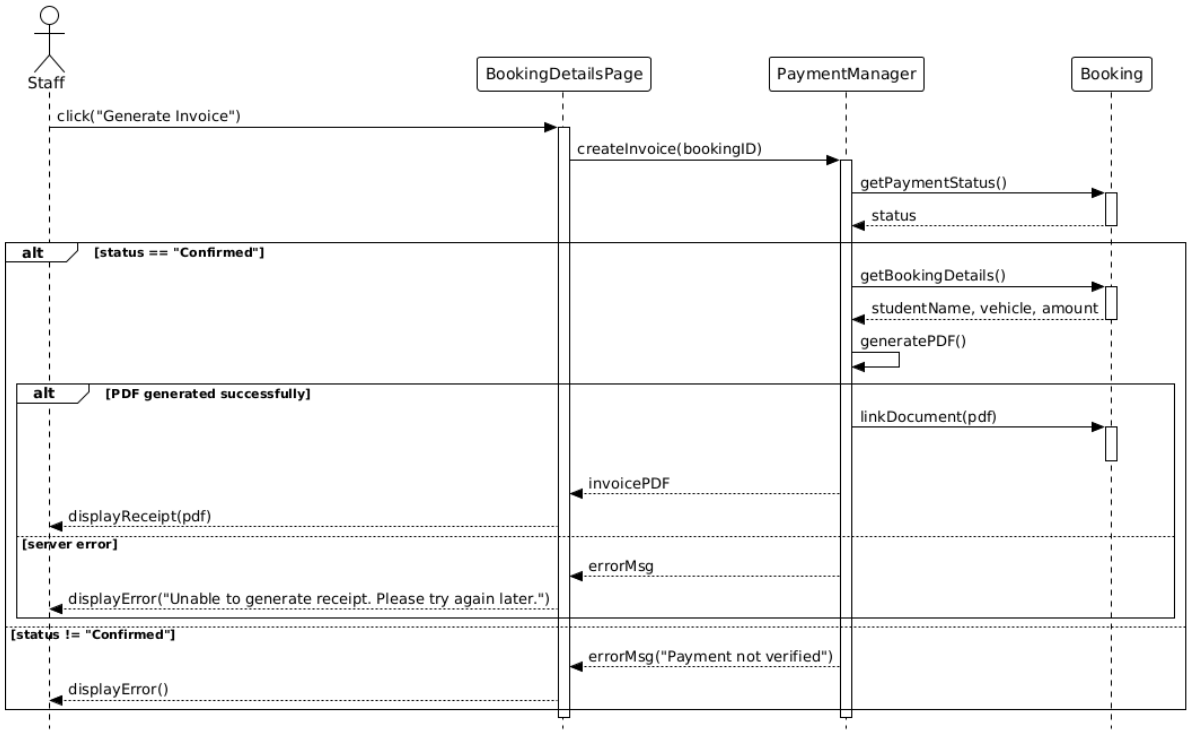

Figure 2.19.1: Sequence Diagram <Generate Invoice/Receipt>

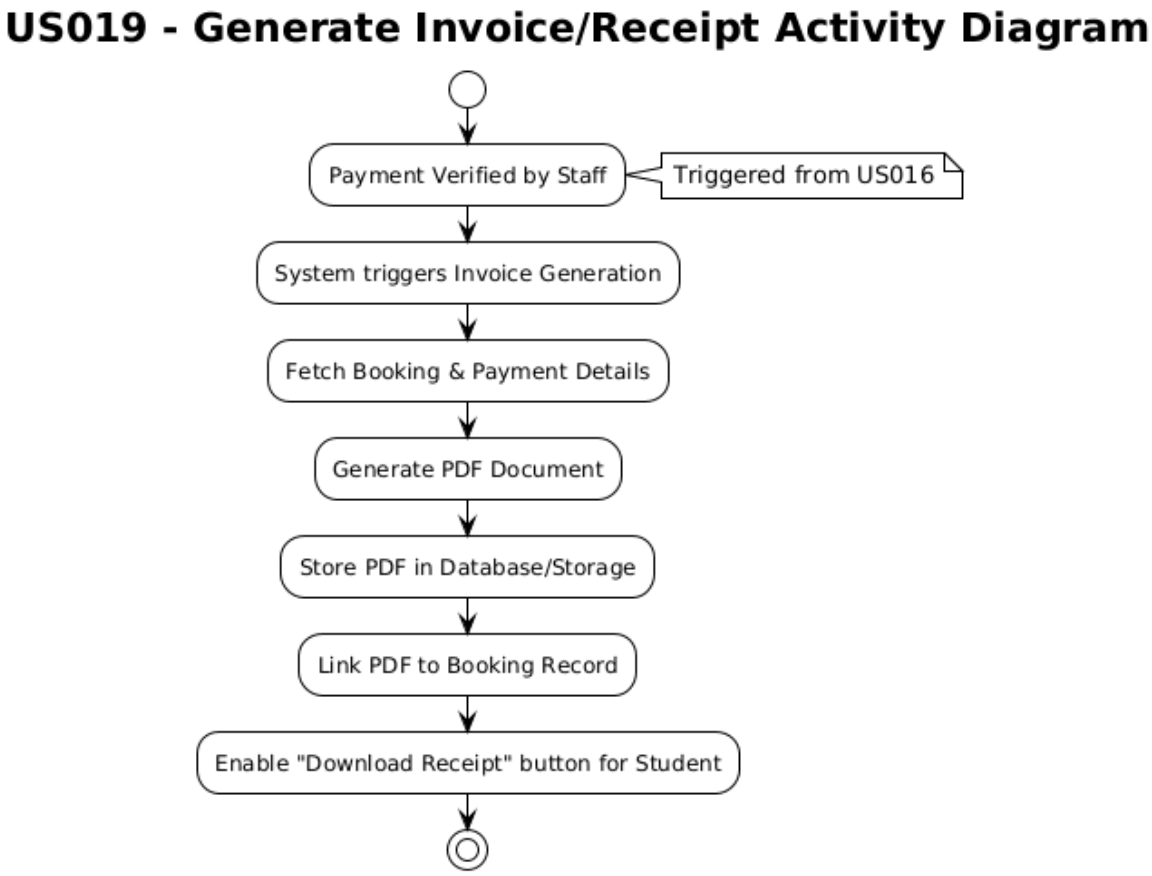

Figure 2.19.2: Sequence Diagram <Generate Invoice/Receipt>

2.4.20 US0020: User Story <Staff> Sprint 2 – Car Booking Module

Table 2.20 depicts User Story Description for <Check Availability> use case of flow events interaction for staff. Figure 2.20.1 shows that the staff required to log into the system and proceed to the dashboard to check all the available cars. Figure 2.20.2 further elaborates on a selectable action provided to staff, corresponding to the flow of events in Table 2.20.

Table 2.20: User Story Description for Check Availability

User story: Check Availability
ID: US020
User Story Description **As a** staff member, **I want** to check car availability **So that** I can ensure the on-time availability is always match to the booking lists.
Flow of events: 1. Staff logs into system 2. Staff navigate to the Availability section. 3. System displays all cars along with the duration of dates that the car is unavailable, labeled as booked, or maintenance. 4. Staff can view booking information for any time slot.
Alternative flow: Staff can use the date filter to check car availability on a specific day.
Acceptance Criteria Postcondition • Staff has complete visibility Precondition • Staff must be logged in • Cars exist in system
Exception flow: If the system cannot load availability cars, show error with retry option.

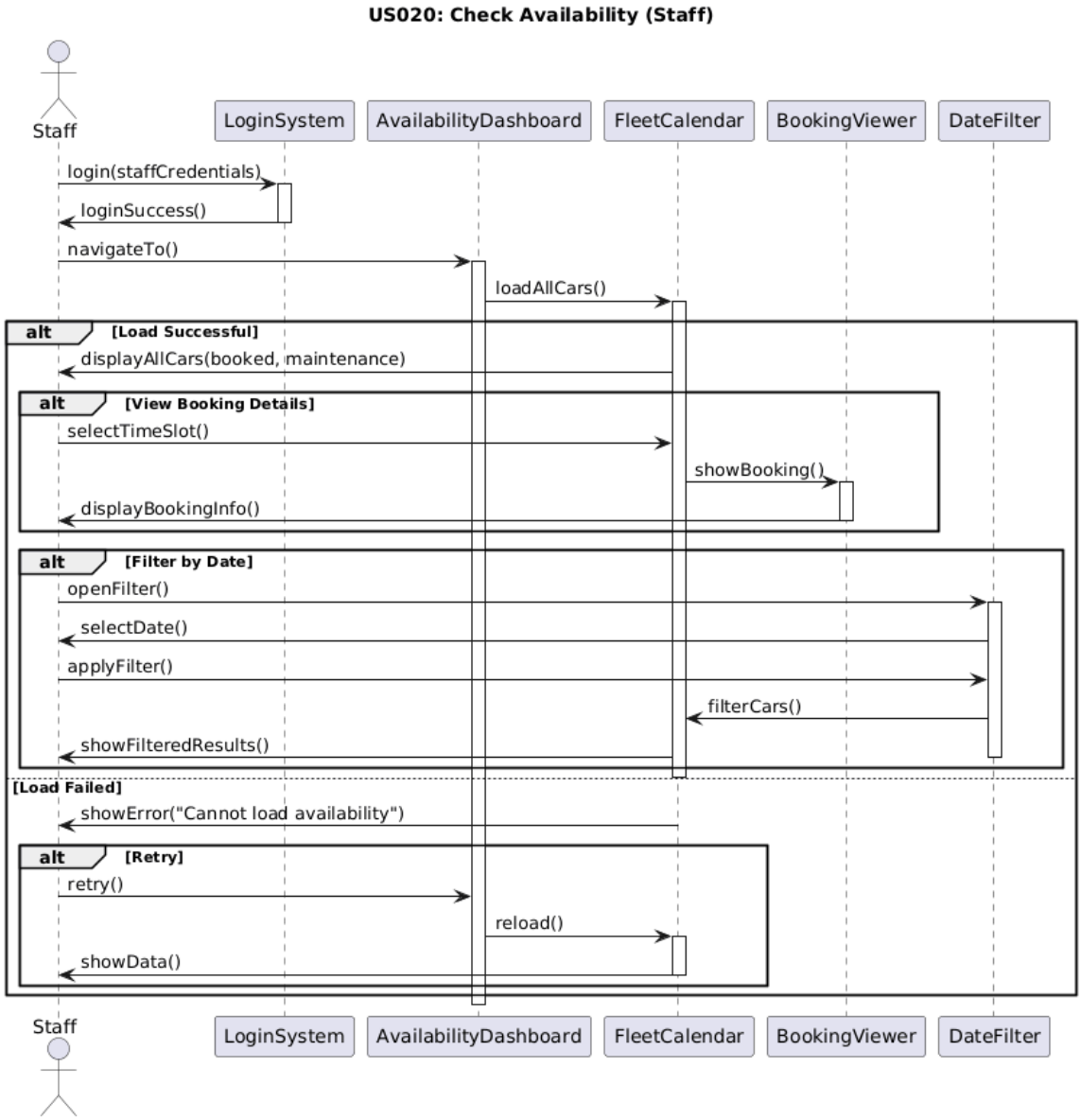

Figure 2.20.1: Sequence Diagram for Check Availability

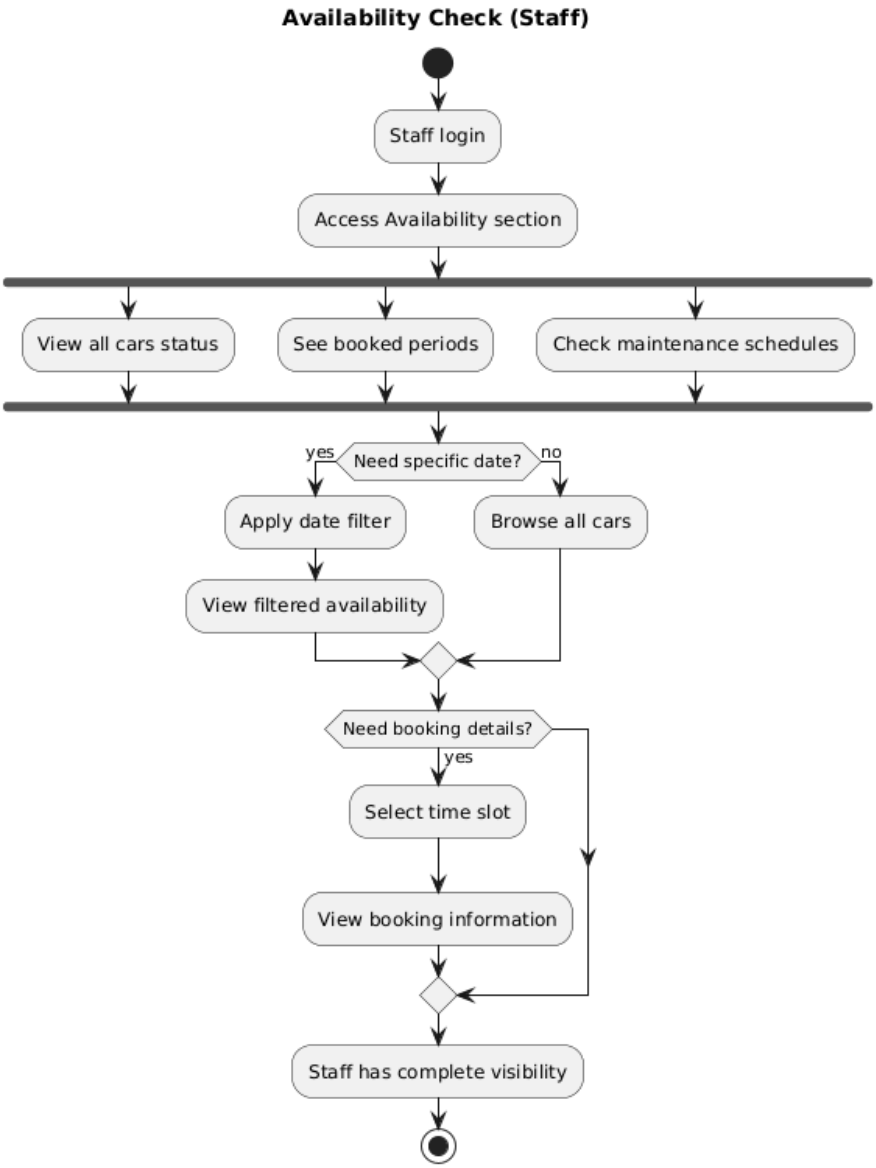

Figure 2.20.2: Activity Diagram for Check Availability

2.4 Performance and Other Requirements

Non-functional requirements describe how the Hasta Car Rental system should operate rather than what it should do. They ensure the system's performance, usability, and quality meet stakeholder expectations. These requirements are categorized into Performance Requirements and Software System Attributes to align with the ISO/IEC/IEEE 29148 standard.

Performance Requirement
Performance requirements define the system’s capability to user requests and handle data efficiently. These requirements include following:

- Response Time: The system shall load the homepage within 3 seconds
- Throughput: The system shall process payment verification updates within 2 seconds after approval
- Capacity: The system shall support a minimum of 10,000 registered users.
- Availability: Users shall be able to access booking and payment features 24/7.

Software System Attributes Requirement

- Software system attributes define the overall qualities or characteristics of the software. These attributes are the foundation on which the software is built, and they include the following:
- Usability: Users can easily register, log in, book cars, view booking history, and cancel reservations with minimal effort.
- Reliability: Booking, cancellation, and payment processes are tested to ensure accuracy and dependability.
- Maintainability: The system is built using modular design principles, making it easy to update, repair, or enhance individual components without affecting the rest of the system.
- Portability: Hasta is designed to operate on multiple platforms, including web browsers and mobile devices.
- Compatibility: The system is compatible with multiple web browsers such as Chrome, Firefox, and Edge and mobile operating systems such as Android and iOS.
- Safety: Data operations, financial transactions, and system actions are controlled to prevent accidental data leak.
- Legal and Regulatory: Hasta complies with relevant laws, regulations, and industry standards regarding data privacy, user protection, and secure financial transactions.
- Environmental: Efficient coding and database management reduce resource usage while maintaining performance.

2.5 Design Constraints

Design Constraints
The development of the Hasta Car Booking System is subject to several organizational, environmental, technical, and regulatory constraints that guide how the system should be designed, implemented, and operated. These constraints ensure that the system will conform to institutional standards, will operate reliably within the environment in which it is placed, and will meet the requirements imposed externally from security, hardware, and software perspectives.

Environmental Constraints
The system will be deployed and accessed within typical indoor environments such as offices, customer accommodations, or campus facilities. Therefore, it must operate reliably under normal indoor conditions. Constraint: The system shall be able to operate reliably in temperatures between 0°C and 40°C and at normal indoor humidity and lighting.

Hardware Constraints
As a web-based system, the system will be accessed on user devices with varying hardware capabilities. It shall not require high-performance hardware to operate effectively. Constraint: The system shall operate smoothly on devices with at least 4GB of RAM, standard dual-core processors, and stable internet connectivity.

Software and Platform Compatibility Constraints
The system shall be compatible with common browsers and operating systems used by customers, staff, and supervisors. Constraint: The system shall support modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, and shall operate on Windows, macOS, Android, and iOS platforms.

Security Constraints
The system deals with sensitive information including personal identification documents, booking records, and proof of payment. Stronger security measures must be put into place to meet confidentiality and integrity requirements. Constraint: All sensitive user data shall be encrypted using AES-256 encryption, and the system shall implement secure authentication and role-based access control. Constraint: The system shall follow secure development best practices to protect against common vulnerabilities like SQL injection, XSS, and unauthorized access.

Organizational and Policy Constraints
The system shall meet Hasta Travel's operational workflow and university policies on data protection and record maintenance. Constraint: The system shall store booking, transaction, and document records in line with the organization's policies on data retention. Constraint: The workflow for the approval of the booking, vehicle management, and reporting shall follow the procedures outlined by the administrative staff and supervisors.

Legal and Regulatory Constraints

The system shall ensure that personal data are processed in accordance with relevant Malaysian data protection guidelines, for example, PDPA. Constraint: The system shall ensure that the users' personal information is kept confidential and accessed only by authorized personnel.

Performance Constraints

The system is anticipated to handle multiple users simultaneously at any given time in the case of peak bookings. Constraint: The system shall be able to support at least 200 concurrent users with a maximum response time of less than 3 seconds for standard operations. Constraint: The system shall update real-time vehicle availability with minimum delay.